final_project_template_2025

December 16, 2025

---

1 Searching for Hot Jupiters in Omega Centauri: A Machine Learning Approach

Author(s): Erin M. Bernthold
Project Category: Astronomy & Astrophysics
Course: Physics 5680, Autumn 2025
Date: December 16, 2025

Image Info: The globular cluster 47 Tucanae

1.1 ###### Image Source: https://www.universeguide.com/cluster/47tucanae

1.2 Abstract

Over the past three decades, the discovery of exoplanets has transformed our understanding of planetary systems, revealing nearly 6000 worlds beyond our own Solar System. Among these are hot Jupiters, which are massive, Jupiter-like planets that orbit extremely close to their stars, making them ideal targets for detection using the transit method. Very little is known about the occurrence of such planets in chemically distinct and densely populated environments like globular clusters. Only one planet has ever been detected in a globular cluster, highlighting a critical gap in our knowledge of how chemical composition and stellar interactions influence planet formation. This project aims to address that gap by applying a machine learning approach to search for transiting planets in the globular cluster Omega Centauri. Building on previous work by Dr. Alison Crisp, who developed and tested transit detection methods using both real and synthetic light curves from 47 Tucanae, this study will employ a random forest (RF) trained on her dataset to identify potential planetary transits in Omega Centauri's light curves. By comparing results between these two clusters, this project seeks to advance our understanding of how element abundances and stellar interactions affect the formation and survival of hot Jupiters in globular cluster environments.

1.3 1. Introduction

The first planet orbiting around a Sun-like star outside of our Solar System was discovered over 30 years ago. Since then, around 6,000 exoplanets have been found using various methods, including the transit method. A transit occurs when a planet passes in front of a star, causing some of the light from the star to dim. It is much easier to find exoplanets that are orbiting close to their star

as the change in the star's brightness can be more easily observed. The overarching goal of this project is to study a specific type of exoplanet called a hot Jupiter. These planets are as massive as our own Jupiter. However, they are found orbiting very close to their star, which is quite strange compared to where our Jupiter is located, but makes them the perfect candidates for the transit method. As a result, there are many theories on how they formed. Some suggest that the hot Jupiter formed much farther from the star, but eventually migrated closer, while others say that the hot Jupiter formed close to its star to begin with. When it comes to planet formation, element abundances play a significant role, especially iron, hydrogen, and the various alpha elements. We don't know the specific elements that affect the formation of a hot Jupiter, so astrophysicists are looking at regions of our galaxy that have different chemical abundances. Globular clusters, which are dense collections of very old stars bounded by gravity, are relatively unexplored regions of space and are chemically different from objects studied by previous surveys, like the Galactic thin disk. If we were to find hot Jupiters in a globular cluster, it could tell us more information on how element abundances affect formation of hot Jupiters and since globular clusters are dense stellar regions, we could learn how stellar interactions affect planet formation as well.

This leads us to our problem. There has only been one occurance of a planet being found in a globular cluster. Since then, there haven't been any more detections, but that doesn't mean we've stopped looking! Astrophysicists have been eyeing two particuar globular clusters, called 47 Tucanae and Omega Centauri, which are some of the most massive and closest clusters to Earth. In particular, 47 Tucanae has been promoted as a planet-hosting cluster, but there has yet to be a significant detection. Dr. Alision Crisp, a post-doctorate at The Ohio State University, has done much work in trying to find planets in 47 Tucanae. Her previous work invoved her creating and injecting transit models into light curves from 47 Tucanae. A search algorithm was then used on the real and synthetic light curves simultaneously where the flagged detections were then subjected to a two-step vetting process. First, the identitfying information of the light curves was removed and then the light curves were mixed together to be inspected by eye. The results of the first round were used to estimate the efficiency in the case of the synthetic light curves and to narrow down the number of candidates that need to be further investigated in the case of the real light curves. Over 10,000 light curves were sent in for human classification, where 39 candidates were identified as potential transits. After another round of detailed analysis, only 18 candidates remained. However, these candidates were also rejected as false positives.

To continue this work, I will be taking a machine learning approach. The previous work that Dr. Crisp has done has provided an ideal training set to seach for new candidates in data that hasn't been used yet, for example, light curves from Omega Centauri. Using the real and synthetic light curves from 47 Tucanae as the input, I will train an algorithm to detect any transits or other variability by processing the light curves through a random forest (RF). The light curves from Omega Centauri will then be used to test this algorithm and we will be able to determine if there are any potential transiting planets in this globular cluster. Whether we find planets or not in Omega Centauri, we will gain valuable insight in how element abundance and stellar interactions affect the formation of planets.

Image Source: https://avanderburg.github.io/tutorial/tutorial.html

---

1.4 2. Related Work

The following papers cover key studies and projects that are relevant to my own topic: - "Using Random Forest Machine Learning Algorithms in Binary Supernovae Classification" by Jonathan Markel and Amanda J. Bayless (2020). - "Photometric Light Curves Classification with Machine Learning" by Gabruseva et al. (2019). - "Using Machine Learning Method for Variable Star Classification Using TESS Sectors 1-57 Data" by Wang et al. (2025)

Extra resources about an alternate approach using a 1D convolutional neural network (CNN): - "LCDC: Bridging Science and Machine Learning for Light Curve Analysis" by Kyselica et al. (2025). - "One-Dimensional Convolutional Neural Networks for Detecting Transiting Exoplanets" by Álvarez et al. (2023). - "Classification of Variable Stars using Convolutional Neural Network" by Bindu et al. (2025). - "Training a Convolutional Neural Network for Exoplanet Classifiation with Transit Photometry" by Juliana Wang (2025). - "Classification of Variable Star Light Curves using Convolutional Neural Network" by Akhmetali et al. (2024).

1.4.1 "Using Random Forest Machine Learning Algorithms in Binary Supernovae Classification"

Citation:
Markel, J., & Bayless, A. J. (2020). Using Random Forest Machine Learning Algorithms in Binary Supernovae Classification. arXiv [Astro-Ph.HE]. Retrieved from http://arxiv.org/abs/1907.00088.

Abstract Summary: In this study, the authors address the need for rapid, automated classification of newly discovered transient objects, focusing on supernovae (SNe). They apply a random forest machine learning algorithm to distinguish between Type Ia and core-collapse supernovae (CCSNe) using full light curves. To evaluate the algorithm's ability both for classification with complete light-curve data and for early-epoch classification, they employ the features of the light curve, such as the shape and magnitude of the peak, to determine the SN type. Their initial investigation demonstrates that, with as few as five data points near peak brightness, the random forest classifier can identify whether an event is a Type Ia or a core-collapse supernova with better than 80% accuracy. However, the current data exhibits biases that should be addressed in future work.

Strengths:
- Demonstrates practical application of random forests to real astronomical time-series classification. - Includes both "full light curve" and "early-epoch" classification contexts. - Defines explicit feature engineering and performance evaluation. - Discusses the preprocessing steps and issues. - Includes a comparison of fitted vs. unfitted data performance.

Weaknesses: - Classification task focuses on supernovae, not transits. - Limited exploration of feature optimization. - Performance focused on specific light-curve shapes and simulation biases. - Focuses on classification accuracy rather than uncertainty quantification.

Overall, Markel & Bayless (2020) provides a useful methodological example of applying random forests to astronomical light curves, including concrete preprocessing choices, feature extraction, and performance evaluation. Their treatment of early-epoch classification is conceptually similar to using partial light curves for transit detection and may help with the development of my own process. However, the specific focus on supernovae and the relatively narrow exploration of feature strategies and model comparisons show that the paper is more valuable as a general methodological reference than a direct template for transit classification.

1.4.2 Paper 2: "Photometric Light Curves Classification with Machine Learning"

Citation:
Gabruseva, T., Zlobin, S., & Wang, P. (2019). Photometric light curves classification with machine learning. arXiv [Astro-Ph.IM]. Retrieved from http://arxiv.org/abs/1909.05032.

Abstract Summary: In preparation for the data influx expected from the Large Synoptic Survey Telescope, this study develops and evaluates an automated method for classifying astronomical photometric light curves using machine learning. The authors implement a classification pipeline that combines gradient boosting decision trees with systematic feature extraction, feature selection, and data augmentation to distinguish among a variety of astronomical object types. This methodology was specifically applied within the context of the Photometric LSST Astronomical Time Series Classification Challenge (PLAsTiCC), where it demonstrated strong performance and achieved one of the top-ranking results in the competition.

Strengths:
- Demonstrates a full classification pipeline on time-domain light curves. - Covers extensive feature exploration and selection. - Uses augmentation to account for small training sets. - Includes the model optimzation and validation procedure. - Reports on the challenges the model has.

Weaknesses: - Classification task focuses on multiple types of astronomical objects, not transits. - Uses gradient-boosted decision trees, not random forests. - The features they extract may not be applicable to transits.

For my project, Gabruseva et al. (2019) is a useful methodological reference rather than a direct template for transit detection. It provides a clear example of a complete machine-learning pipeline for photometric light-curve classification, including feature extraction, feature selection, data augmentation, and model validation using decision-tree–based methods, all of which are applicable to a random forest approach. However, the study does not address transit signals specifically, relies on simulated multi-class datasets tailored to PLAsTiCC object types, and focuses on gradient-boosted trees rather than random forests. As a result, while the general strategies for handling time-series data and class imbalance are informative, additional transit-specific features and validation on real injected transit data will be necessary to fully adapt the approach to this project.

1.4.3 Paper 3: "Using Machine Learning Method for Variable Star Classification Using TESS Sectors 1-57 Data"

Citation:
Wang, L.-H., Li, K., Gao, X., Guo, Y.-N., & Sun, G.-Y. (2025). Using machine learning method for variable star classification using the TESS Sectors 1-57 data. arXiv [Astro-Ph.SR]. Retrieved from http://arxiv.org/abs/2504.00347.

Abstract Summary: The Transiting Exoplanet Survey Satellite (TESS) has conducted over four years of photometric observations, producing approximately 1,050,000 light curves at 2-minute cadence. By cross-matching these light curves with Gaia's variable star catalogue, the authors constructed a labeled dataset for supervised learning. They used a random forest classifier to perform variable star classification, designing distinct classification processes for subclasses such as eclipsing binaries (EA, EW), Cepheids (CEP), Delta Scuti stars (DSCT), RR Lyrae (RRab, RRc), and rotational variables (ROT). Visual inspection was then applied to all candidates to ensure catalog reliability and accuracy. After classification and vetting, the authors identified final counts for each subclass and discovered 14,092 new variable stars in the TESS Sectors 1–57 data.

Strengths:
- Uses a real, large-scale dataset. - Invloves a sub-class specific classification. - Includes a visiual vetting process for quality control.

Weaknesses: - Classification task focuses on variable stars, not transits. - No detection features or processing techniques relevant to detection features. - Visual inspection is time-consuming.

Wang et al. (2025) is a valuable reference because it applies a random forest classifier to a large real light-curve dataset and demonstrates practical steps for supervised classification, label vetting, and subclass-specific processing. However, because its focus is on variable star classification rather than transit versus non-transit signals, and it does not explore transit-specific features or detection strategies, I would need to adapt and extend its methods to suit the specific characteristics of transit signal classification.

1.5 3. Ethical Considerations

One thing that should be noted is the potential bias in the training dataset. From the previous work done by Dr. Crisp, potential transits were flagged through the human classification process. The majority of the light curves were analyzed by two people, Dr. Crisp and her advisor Dr. Matthew Penny. Since they both knew that there were real light curves and light curves with injected transits in the sample, this may have biased the result of the training set. When analyzing the results of my project, this fact should be considered.

From what I can see, there is no clear potential misuse of the algorithm that I will create since it will only have the capability to determine if a light curve has a transiting planet or not. Additonally, if this work is successful, it will speed up the process of finding potential transits in different areas of our Universe. This will be a great help due to the influx of newly disovered astronomical objects, and for those yet to be discovered due to the creation of new telescopes and instruments, such as the Nancy Grace Roman Space Telescope set to launch in 2026-27.

Also note, Google Gemini was used for troubleshooting code.

1.6 4. Project Setup and Imports

1.6.1 1.1. Key Libraries

Below is a brief description of the primary Python packages that will be used in this project.

- **pandas:** Used for data manipulation and analysis. It provides powerful data structures, like the DataFrame, for handling and exploring structured data.
- **numpy:** The fundamental package for scientific computing in Python. It provides support for large, multi-dimensional arrays and matrices, along with a collection of mathematical functions to operate on these arrays.
- **matplotlib & seaborn:** These are libraries for data visualization. Matplotlib is a comprehensive library for creating static visualizations, while Seaborn provides a high-level interface for drawing attractive and informative statistical graphics.

- **Plotly:** A graphing library used to create interactive, publication-quality visualizations. This is especially useful for creating dynamic plots that allow for zooming, panning, and hovering to inspect data points.
- **scikit-learn:** A primary machine learning library that provides simple and efficient tools for data mining and data analysis, including a wide range of classification, regression, and clustering algorithms.
- **TensorFlow:** An end-to-end open-source platform for machine learning, specializing in deep learning. It is used for building and training neural networks for tasks like image classification, natural language processing, and more.

[8]:
```
# Import all necessary libraries here
import tensorflow as tf
from tensorflow import keras
from keras import models
from keras import layers
import sys
import os

import plotly
import plotly.express as px
import plotly.io as pio
pio.renderers.default = 'notebook+pdf'

import tarfile
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

import sklearn
from sklearn import metrics
from sklearn.metrics import auc
from sklearn.model_selection import train_test_split
from sklearn.model_selection import StratifiedKFold
from sklearn.metrics import confusion_matrix, accuracy_score,␣
 ↪ConfusionMatrixDisplay
from sklearn.ensemble import RandomForestClassifier
from sklearn.utils import shuffle

from tabulate import tabulate

print("Libraries imported successfully.")
```

```
2025-12-16 16:48:03.609884: I tensorflow/core/platform/cpu_feature_guard.cc:210]
This TensorFlow binary is optimized to use available CPU instructions in
performance-critical operations.
To enable the following instructions: AVX2 AVX512F FMA, in other operations,
rebuild TensorFlow with the appropriate compiler flags.
```

```
Libraries imported successfully.
```

1.6.2 4.2. Version Information

[9]:
```
# Print version information
print(f"Python version: {sys.version}")
print(f"pandas version: {pd.__version__}")
print(f"numpy version: {np.__version__}")
print(f"seaborn version: {sns.__version__}")
print(f"plotly version: {plotly.__version__}")
print(f"scikit-learn version: {sklearn.__version__}")
print(f"tensorflow version: {tf.__version__}")
```

```
Python version: 3.9.18 (main, Sep  4 2025, 00:00:00)
[GCC 11.4.1 20231218 (Red Hat 11.4.1-3)]
pandas version: 2.3.1
numpy version: 2.0.2
seaborn version: 0.13.2
plotly version: 6.3.0
scikit-learn version: 1.6.1
tensorflow version: 2.20.0
```

1.7 5. The Dataset

For the training set, I will be using real and synthetic light curves acquired from the Dark Energy Camera (DECam). DECam consists of 62 charge-coupled devices (CCDs), which are integrated circuits containing an array of linked capacitors that are used for direct imaging. I have light curves for each of these chips, however, due to the size of the files, I will only be using data from S11, S12, S13, S14, and S15. These chips are located near the center of the instrument. Additionally, DECam uses 8 photometric filters, including the r-band (568-716nm) and z-band (850-1002nm).

Image Source 1: https://noirlab.edu/science/programs/ctio/instruments/Dark-Energy-Camera/characteristics

Image Source 2: https://noirlab.edu/science/programs/ctio/filters/Dark-Energy-Camera

Inside each of the files is the following data:
| Column Name | Description | |————|————-| | MJD | Modified Julian Date (MJD) | | flux_diff_1 | Difference flux 1 | | flux_diff_1_err | Error of difference flux 1 | | flux_diff_2 | Difference flux 2 | | flux_diff_2_err | Error of difference flux 2 | | aobc | $\frac{\lvert\sum_i \mathrm{im}_i\,\mathrm{psf}_i\rvert}{\sqrt{(\sum_i \mathrm{im}_i^2)(\sum_i \mathrm{psf}_i^2)}}$; normalized cross-correlation (cosine similarity) between image and PSF | | sky_flux_1 | Sky flux 1 | | sky_flux_2 | Sky flux 2 | | sky | Sky value | | m_sky | Mean sky value | | n_sky_pixels | Number of sky pixels | | chip_center_BJD | Barycentric Julian Date for chip center | | flux_diff_1_mag | Magnitude derived from difference flux 1 | | flux_diff_1_mag_err | Error on magnitude from difference flux 1 | | flux_diff_2_mag | Magnitude derived from difference flux 2 | | flux_diff_2_mag_err | Error on magnitude from difference flux 2 | | PSF_to_FWHM_radius | PSF radius relative to the FWHM | | sky_background_counts | Sky background counts (larger in z band than r band) |

| frame_zeropoint_mag | Magnitude zeropoint of the frame | | PSF_ellipticity | PSF ellipticity | | airmass | Airmass of the observation | | image_name | Image file name |

To get to the data within these files, they will need to be extracted from tar files and then complied into a single data frame. From there, the data will be randomized to decrease any bias in the data. Since the data files had no column headers originally, they will be added based on the data information above, and two more headers will be added. One that specifies which file the data is from and another that tells whether this light curve has a transit, either 'YES' or 'NO'. The transit header will then be converted into a bianry format, 1 for 'YES' and 0 for 'NO'.

I may need to be more careful with this data due to the use of the two optical filters, r and z. Right now, I'm compiling everything in one data frame, but this might be the wrong course of action. Or, I may have to consider the features of the r-band filter different from the z-band filter, essentially doubling the number of features I can consider for my algorthim.

1.7.1 5.1. Load and Preprocess Data

Extract the data from the tar files This was done three times for the r-band, z-band, and synthetic light curves, respectively. The same headers were applied to each. 'source_file' and 'transit' were appended to the data frame. There are also some files that have been dereddened. We will filter these out since the synthetic light curves have not been dereddened. Additionally, the data frames will be saved as CSV files to make it easier to read in later. This makes it so the code to extract the data only needs to be ran once. To run it again, simply uncomment the code and change the `tarfilename` to match the tar file whose data you want to extract.

[6]:
```
# 1. Setup: Define the column headers that will be applied to the data files,␣
 ↪the tar filename,
# and an empty list to hold all the files wihtin this tar file.
#column_headers = [
#    'MJD', 'flux_diff_1', 'flux_diff_1_err', 'flux_diff_2', 'flux_diff_2_err',
#    'aobc', 'sky_flux_1', 'sky_flux_2', 'sky', 'm_sky', 'n_sky_pixels',
#    'chip_center_BJD', 'flux_diff_1_mag', 'flux_diff_1_mag_err',␣
 ↪'flux_diff_2_mag',
#    'flux_diff_2_mag_err', 'PSF_to_FWHM_radius', 'sky_background_counts',
#    'frame_zeropoint_mag', 'PSF_ellipticity', 'airmass', 'image_name'
#]

#tar_filename = '/fs/scratch/PAS2038/bernthold306/S11_data/
 ↪MISHAPS_F47T_S11_r_2019to2022_lc.tar.gz'
#all_files = []

# 2. Open the tar file in read mode
#with tarfile.open(tar_filename, 'r') as tar:

    # Iterate through every file inside the tar
#    num=0
#    print('successfully opened')
#    for member in tar.getmembers():
```

```
        # Check if it is an .lc file AND NOT a .dtr file
        # (The 'not .dtr' is technically redundant but good for safety)
#         if member.name.endswith('.lc') and not member.name.endswith('.dtr')␣
 ↪and member.isfile():

            # Only print if we are actually keeping it
#             print(f"Processing: {member.name}")

            # Extract the file object (in memory)
#             f = tar.extractfile(member)

#             if f:
                # Read into Pandas with names
#                 df = pd.read_csv(f, header=None, sep='\s+',␣
 ↪names=column_headers)

                # Add the source column and transit column
#                 df['source_file'] = member.name
#                 df['transit'] = 'NO'
                # Append to list
#                 all_files.append(df)

                # Counter check (optional)
#                 num += 1
#                 if num > 10:
#                     break
# 3. Combine into a single data frame
#if all_files:
#    r_filt_df = pd.concat(all_files, ignore_index=True)

    # Count total rows before filtering
#    total_rows = len(r_filt_df)

    # Identify rows where 'image_name' ends with .dtr
    # na=False handles cases where image_name might be missing/NaN
#    dtr_mask = r_filt_df['source_file'].str.endswith('.dtr', na=False)
#    dtr_count = dtr_mask.sum()

#    print(f"Total rows loaded: {total_rows}")
#    print(f"Rows with .dtr source_file found: {dtr_count}")

    # Apply the filter (Keep rows that are NOT .dtr)
#    if dtr_count > 0:
#        r_filt_df = r_filt_df[~dtr_mask]
#        print(f"Removed {dtr_count} rows. Final row count: {len(r_filt_df)}")
#    else:
#        print("No .dtr rows found to remove.")
```

```
#    print(r_filt_df.head())

#else:
#    print("No .lc files found in the archive.")

# 4. Save as a CSV file
#basename = os.path.basename(tar_filename_)
#print(basename)
#r_filt_df.to_csv(f'/fs/scratch/PAS2038/bernthold306/test_{basename}.
 ↪csv',index=False)
#print("Saved Successfully!")
#print(len(r_filt_df))
```

Combine the data into a single data frame Since I've already ran the code above multiple times, I will read in the CSV files here and compile them into a data frame. You can definitely do this process with a for loop instead of individually, but I just decided to do the brute force method since I might need to add more data files later from different DECam chips.

[11]:
```
# S11 Data
S11_r_df = pd.read_csv("test_MISHAPS_F47T_S11_r_2019to2022_lc.tar.gz.csv")
#print(S11_r_df.head(5))

S11_z_df = pd.read_csv("test_MISHAPS_F47T_S11_z_2019to2022_lc.tar.gz.csv")
#print(S11_z_df.head(5))

S11_injection_df = pd.read_csv("test_MISHAPS_F47T_S11_2019to2022_injection_lcs.
 ↪tar.csv")
#print(S11_injection_df.head(5))
```

[12]:
```
# S12 Data
S12_injection_df = pd.read_csv("test_MISHAPS_F47T_S12_2019to2022_injection_lcs.
 ↪tar.csv")
#print(S12_injection_df.head(5))

# S13 Data
S13_injection_df = pd.read_csv("test_MISHAPS_F47T_S13_2019to2022_injection_lcs.
 ↪tar.csv")
#print(S13_injection_df.head(5))
```

[13]:
```
# S14 Data
S14_r_df = pd.read_csv("test_MISHAPS_F47T_S14_r_2019to2022_lc.tar.gz.csv")
#print(S14_r_df.head(5))

S14_z_df = pd.read_csv("test_MISHAPS_F47T_S14_z_2019to2022_lc.tar.gz.csv")
#print(S14_z_df.head(5))
```

```
S14_injection_df = pd.read_csv("test_MISHAPS_F47T_S14_2019to2022_injection_lcs.
 ↪tar.csv")
#print(S14_injection_df.head(5))
```

[14]:
```
# S15 Data
S15_r_df = pd.read_csv("test_MISHAPS_F47T_S15_r_2019to2022_lc.tar.gz.csv")
#print(S15_r_df.head(5))

S15_z_df = pd.read_csv("test_MISHAPS_F47T_S15_z_2019to2022_lc.tar.gz.csv")
#print(S15_z_df.head(5))

S15_injection_df = pd.read_csv("test_MISHAPS_F47T_S15_2019to2022_injection_lcs.
 ↪tar.csv")
#print(S15_injection_df.head(5))
```

Now, I'm going to combine these into one data frame. I will be doing this in a roundabout way so I can troubleshoot quickly and be more mindful of the process. Additionally, I may want to only use one part of the data frame for something, so it's best to have them in separate data frames instead of combining them into one immediately.

[17]:
```
# Combine the the files using the r-band into one data frame
r_filt_df = pd.concat([S11_r_df, S14_r_df, S15_r_df], ignore_index=True)
print(len(r_filt_df))

# Combine the the files using the z-band into one data frame
z_filt_df = pd.concat([S11_z_df, S14_z_df, S15_z_df], ignore_index=True)
print(len(z_filt_df))

# Combine the r- and z-band data frames into one
real_df = pd.concat([r_filt_df, z_filt_df], ignore_index=True)
print(len(real_df ))
```

```
39567
21681
61248
```

[18]:
```
# Do the same for the injection data frames.
injection_df = pd.concat([S11_injection_df, S12_injection_df, S13_injection_df,␣
 ↪S14_injection_df, S15_injection_df], ignore_index=True)
print(len(injection_df))
```

```
53355
```

[19]:
```
# Combine everything in one data frame
tuc47_lc_df = pd.concat([real_df, injection_df], ignore_index=True)
print(len(tuc47_lc_df))
```

```
114603
```

Convert the 'transit' column into a binary format

[28]:
```
# Convert the 'transit' column to a binary format. 1 for 'YES', 0 for 'NO'.
# Create a dictionary to define which number goes to which result
transit_dc = {'YES': 1, 'NO': 0}

tuc47_lc_df['transit_value'] = tuc47_lc_df['transit'].map(transit_dc)
#print(tuc47_lc_df['transit_value'])
```

Shuffle the data

[30]:
```
shuffled_tuc47_lc_df = shuffle(tuc47_lc_df)
#print(shuffled_tuc47_lc_df['transit'])
#print(shuffled_tuc47_lc_df['transit_value'])
```

Remove NaNs from the data frame, if there are any

[38]:
```
shuffled_tuc47_lc_df = shuffled_tuc47_lc_df.dropna()
```

1.7.2 5.2. Exploratory Data Analysis (EDA)

Determine the best feature correlation

[36]:
```
features = ['transit_value', 'MJD', 'flux_diff_1', 'flux_diff_1_err',␣
 ↪'flux_diff_2', 'flux_diff_2_err',
    'aobc', 'sky_flux_1', 'sky_flux_2', 'sky', 'm_sky', 'n_sky_pixels',
    'chip_center_BJD', 'flux_diff_1_mag', 'flux_diff_1_mag_err',␣
 ↪'flux_diff_2_mag',
    'flux_diff_2_mag_err', 'PSF_to_FWHM_radius', 'sky_background_counts',
    'frame_zeropoint_mag', 'PSF_ellipticity', 'airmass']

# Calculate correlation
feature_corr = shuffled_tuc47_lc_df[features].corr()
feature_corr
```

[36]:

	transit_value	MJD	flux_diff_1	flux_diff_1_err	\
transit_value	1.000000	0.006143	0.004796	0.217689	
MJD	0.006143	1.000000	-0.004695	0.101334	
flux_diff_1	0.004796	-0.004695	1.000000	-0.010092	
flux_diff_1_err	0.217689	0.101334	-0.010092	1.000000	
flux_diff_2	0.002661	-0.004740	0.932395	-0.006535	
flux_diff_2_err	0.229507	0.096998	-0.010176	0.999517	
aobc	0.097891	-0.037954	-0.037253	0.053387	
sky_flux_1	-0.020355	0.011517	0.102993	-0.026368	
sky_flux_2	-0.019739	0.011046	0.138919	-0.026420	
sky	-0.001994	0.002787	-0.016948	0.000570	
m_sky	0.002535	0.006586	0.000764	-0.006268	
n_sky_pixels	NaN	NaN	NaN	NaN	
chip_center_BJD	0.006143	1.000000	-0.004695	0.101334	
flux_diff_1_mag	-0.774546	-0.010084	0.034369	-0.287674	
flux_diff_1_mag_err	-0.011177	-0.000116	0.000596	0.004740	

flux_diff_2_mag	-0.774717	-0.009858	0.035067	-0.290105
flux_diff_2_mag_err	-0.155095	0.027187	0.004201	0.044634
PSF_to_FWHM_radius	-0.053048	-0.144801	-0.015841	0.086529
sky_background_counts	0.075375	0.316816	-0.035271	0.342160
frame_zeropoint_mag	0.000032	-0.024957	-0.000810	0.096581
PSF_ellipticity	0.014391	-0.158180	-0.025805	0.225769
airmass	0.006490	0.358915	-0.005777	0.115651

	flux_diff_2	flux_diff_2_err	aobc	sky_flux_1	\
transit_value	0.002661	0.229507	0.097891	-0.020355	
MJD	-0.004740	0.096998	-0.037954	0.011517	
flux_diff_1	0.932395	-0.010176	-0.037253	0.102993	
flux_diff_1_err	-0.006535	0.999517	0.053387	-0.026368	
flux_diff_2	1.000000	-0.006650	-0.027126	0.046630	
flux_diff_2_err	-0.006650	1.000000	0.055691	-0.026356	
aobc	-0.027126	0.055691	1.000000	-0.038946	
sky_flux_1	0.046630	-0.026356	-0.038946	1.000000	
sky_flux_2	0.083986	-0.026394	-0.039376	0.998450	
sky	-0.021775	0.000591	-0.005049	0.075167	
m_sky	-0.001576	-0.006316	-0.000337	0.033539	
n_sky_pixels	NaN	NaN	NaN	NaN	
chip_center_BJD	-0.004740	0.096998	-0.037954	0.011517	
flux_diff_1_mag	0.028778	-0.304646	-0.149261	0.029237	
flux_diff_1_mag_err	0.000391	0.004552	0.006643	0.000556	
flux_diff_2_mag	0.030333	-0.307143	-0.152454	0.029326	
flux_diff_2_mag_err	0.003098	0.041964	-0.012915	0.003803	
PSF_to_FWHM_radius	-0.010682	0.082140	0.125132	-0.021546	
sky_background_counts	-0.035404	0.336491	0.002207	0.006961	
frame_zeropoint_mag	-0.000295	0.095387	0.088150	-0.000816	
PSF_ellipticity	-0.016890	0.221946	0.162901	-0.034212	
airmass	-0.009156	0.112728	-0.032496	0.007166	

	sky_flux_2	sky	…	chip_center_BJD	\
transit_value	-0.019739	-0.001994	…	0.006143	
MJD	0.011046	0.002787	…	1.000000	
flux_diff_1	0.138919	-0.016948	…	-0.004695	
flux_diff_1_err	-0.026420	0.000570	…	0.101334	
flux_diff_2	0.083986	-0.021775	…	-0.004740	
flux_diff_2_err	-0.026394	0.000591	…	0.096998	
aobc	-0.039376	-0.005049	…	-0.037954	
sky_flux_1	0.998450	0.075167	…	0.011517	
sky_flux_2	1.000000	0.074918	…	0.011046	
sky	0.074918	1.000000	…	0.002787	
m_sky	0.033979	0.381630	…	0.006586	
n_sky_pixels	NaN	NaN	…	NaN	
chip_center_BJD	0.011046	0.002787	…	1.000000	
flux_diff_1_mag	0.029346	0.002928	…	-0.010084	

flux_diff_1_mag_err	0.000554	0.000028	…	-0.000116
flux_diff_2_mag	0.029526	0.002927	…	-0.009858
flux_diff_2_mag_err	0.003755	0.000392	…	0.027187
PSF_to_FWHM_radius	-0.022182	-0.007358	…	-0.144801
sky_background_counts	0.005774	0.007400	…	0.316816
frame_zeropoint_mag	-0.001330	-0.001097	…	-0.024957
PSF_ellipticity	-0.035259	-0.013910	…	-0.158180
airmass	0.006940	0.000168	…	0.358914

	flux_diff_1_mag	flux_diff_1_mag_err	flux_diff_2_mag	\
transit_value	-0.774546	-0.011177	-0.774717	
MJD	-0.010084	-0.000116	-0.009858	
flux_diff_1	0.034369	0.000596	0.035067	
flux_diff_1_err	-0.287674	0.004740	-0.290105	
flux_diff_2	0.028778	0.000391	0.030333	
flux_diff_2_err	-0.304646	0.004552	-0.307143	
aobc	-0.149261	0.006643	-0.152454	
sky_flux_1	0.029237	0.000556	0.029326	
sky_flux_2	0.029346	0.000554	0.029526	
sky	0.002928	0.000028	0.002927	
m_sky	-0.003031	-0.000052	-0.003036	
n_sky_pixels	NaN	NaN	NaN	
chip_center_BJD	-0.010084	-0.000116	-0.009858	
flux_diff_1_mag	1.000000	0.042937	0.999387	
flux_diff_1_mag_err	0.042937	1.000000	0.026078	
flux_diff_2_mag	0.999387	0.026078	1.000000	
flux_diff_2_mag_err	0.278591	0.140451	0.268210	
PSF_to_FWHM_radius	0.071542	0.004491	0.070442	
sky_background_counts	-0.112037	0.001681	-0.112777	
frame_zeropoint_mag	0.001825	0.024687	-0.000224	
PSF_ellipticity	-0.021708	0.006903	-0.023841	
airmass	-0.011950	-0.001181	-0.011811	

	flux_diff_2_mag_err	PSF_to_FWHM_radius	\
transit_value	-0.155095	-0.053048	
MJD	0.027187	-0.144801	
flux_diff_1	0.004201	-0.015841	
flux_diff_1_err	0.044634	0.086529	
flux_diff_2	0.003098	-0.010682	
flux_diff_2_err	0.041964	0.082140	
aobc	-0.012915	0.125132	
sky_flux_1	0.003803	-0.021546	
sky_flux_2	0.003755	-0.022182	
sky	0.000392	-0.007358	
m_sky	-0.000766	0.000736	
n_sky_pixels	NaN	NaN	
chip_center_BJD	0.027187	-0.144801	

flux_diff_1_mag	0.278591	0.071542
flux_diff_1_mag_err	0.140451	0.004491
flux_diff_2_mag	0.268210	0.070442
flux_diff_2_mag_err	1.000000	0.018760
PSF_to_FWHM_radius	0.018760	1.000000
sky_background_counts	0.049718	-0.234513
frame_zeropoint_mag	0.012292	0.156628
PSF_ellipticity	0.019122	0.671857
airmass	0.019448	-0.059677

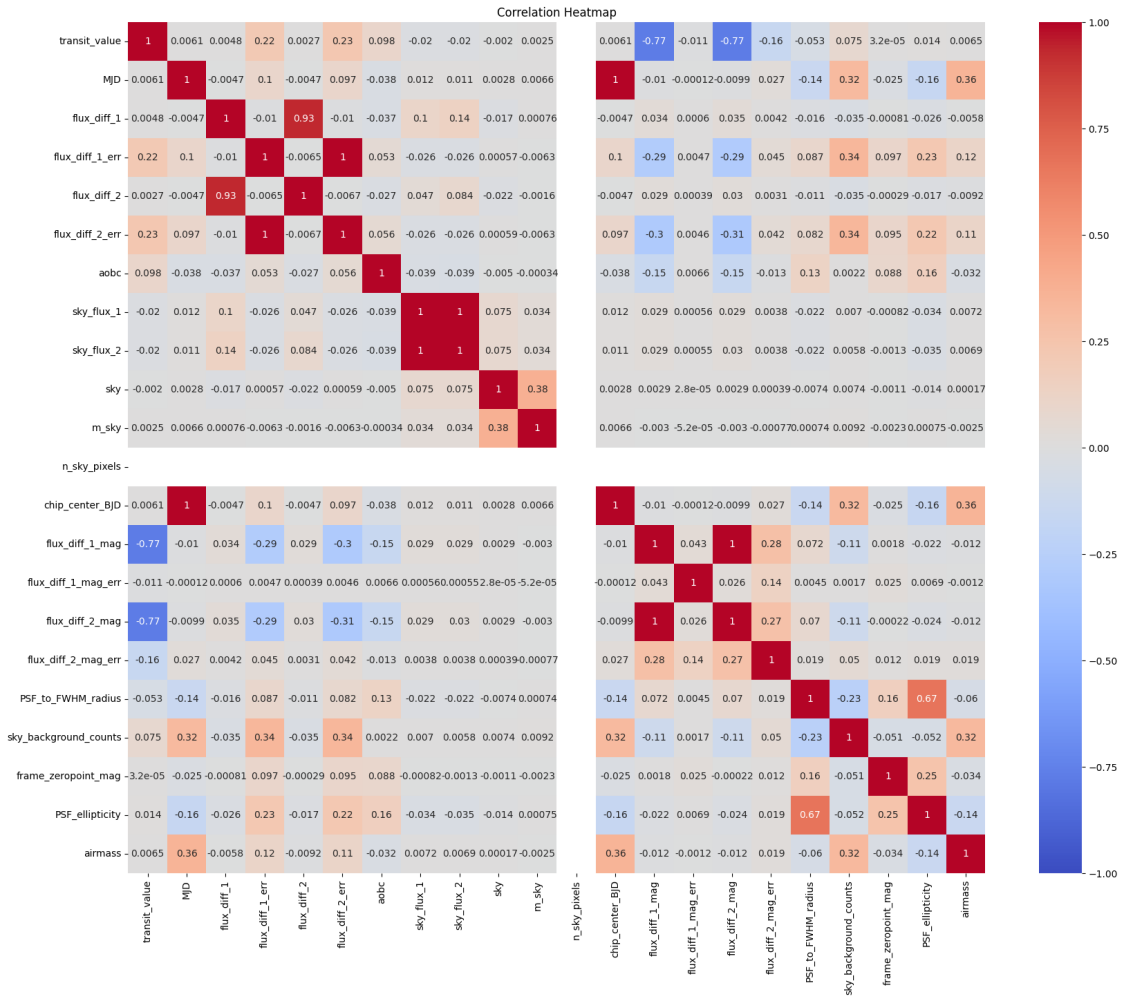
	sky_background_counts	frame_zeropoint_mag	\
transit_value	0.075375	0.000032	
MJD	0.316816	-0.024957	
flux_diff_1	-0.035271	-0.000810	
flux_diff_1_err	0.342160	0.096581	
flux_diff_2	-0.035404	-0.000295	
flux_diff_2_err	0.336491	0.095387	
aobc	0.002207	0.088150	
sky_flux_1	0.006961	-0.000816	
sky_flux_2	0.005774	-0.001330	
sky	0.007400	-0.001097	
m_sky	0.009177	-0.002305	
n_sky_pixels	NaN	NaN	
chip_center_BJD	0.316816	-0.024957	
flux_diff_1_mag	-0.112037	0.001825	
flux_diff_1_mag_err	0.001681	0.024687	
flux_diff_2_mag	-0.112777	-0.000224	
flux_diff_2_mag_err	0.049718	0.012292	
PSF_to_FWHM_radius	-0.234513	0.156628	
sky_background_counts	1.000000	-0.050948	
frame_zeropoint_mag	-0.050948	1.000000	
PSF_ellipticity	-0.051903	0.254752	
airmass	0.321503	-0.034250	

	PSF_ellipticity	airmass
transit_value	0.014391	0.006490
MJD	-0.158180	0.358915
flux_diff_1	-0.025805	-0.005777
flux_diff_1_err	0.225769	0.115651
flux_diff_2	-0.016890	-0.009156
flux_diff_2_err	0.221946	0.112728
aobc	0.162901	-0.032496
sky_flux_1	-0.034212	0.007166
sky_flux_2	-0.035259	0.006940
sky	-0.013910	0.000168
m_sky	0.000750	-0.002462
n_sky_pixels	NaN	NaN

```
chip_center_BJD              -0.158180  0.358914
flux_diff_1_mag              -0.021708 -0.011950
flux_diff_1_mag_err           0.006903 -0.001181
flux_diff_2_mag              -0.023841 -0.011811
flux_diff_2_mag_err           0.019122  0.019448
PSF_to_FWHM_radius            0.671857 -0.059677
sky_background_counts        -0.051903  0.321503
frame_zeropoint_mag           0.254752 -0.034250
PSF_ellipticity               1.000000 -0.144141
airmass                      -0.144141  1.000000

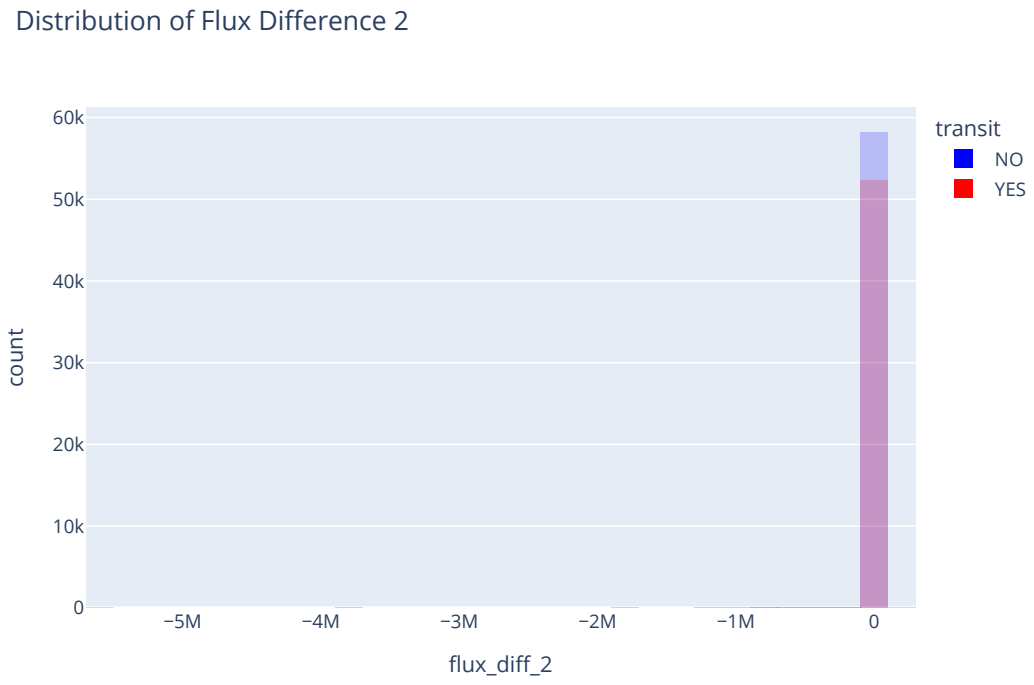
[22 rows x 22 columns]
```

[41]:
```
# Plot heatmap
plt.figure(figsize=(20,16))
sns.heatmap(feature_corr, annot=True, cmap='coolwarm', vmin=-1, vmax=1)
plt.title("Correlation Heatmap")
plt.show()
```

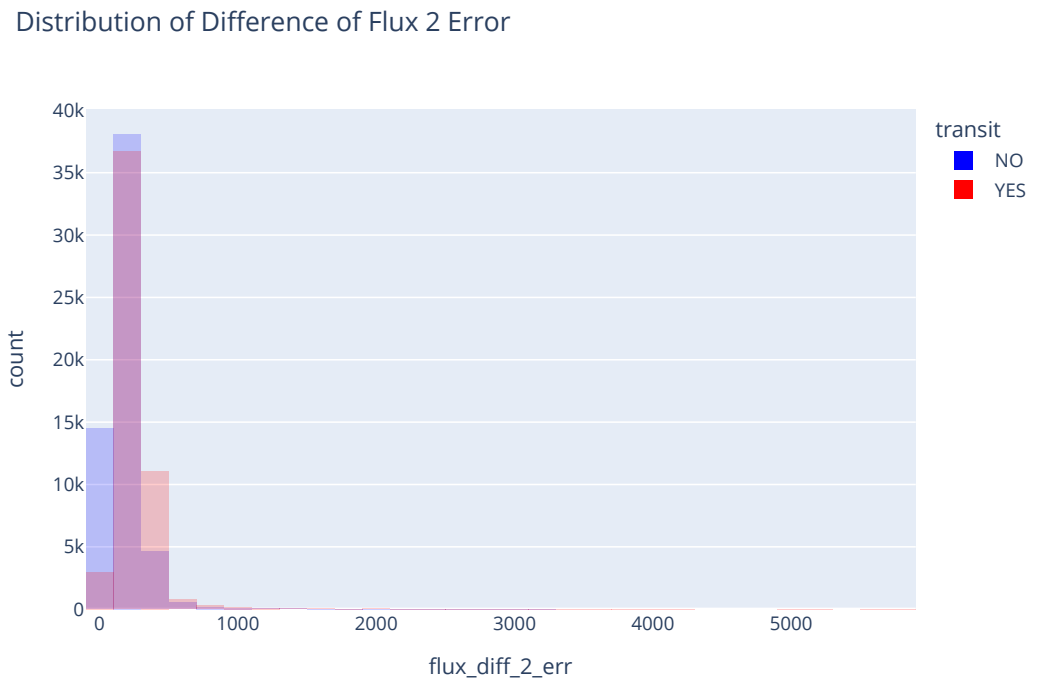


Create histograms of some of those features

[43]:
```
custom_colors = {
    'YES': 'red',    # Map value 1 to red
    'NO': 'blue'    # Map value 0 to blue
}
fig = px.histogram(
        shuffled_tuc47_lc_df,
        x= 'flux_diff_2',
        color='transit',
        barmode='overlay',
        nbins=50,
        title=f"Distribution of Flux Difference 2",
        color_discrete_map=custom_colors,
        opacity=0.2
    )
fig.show()
```

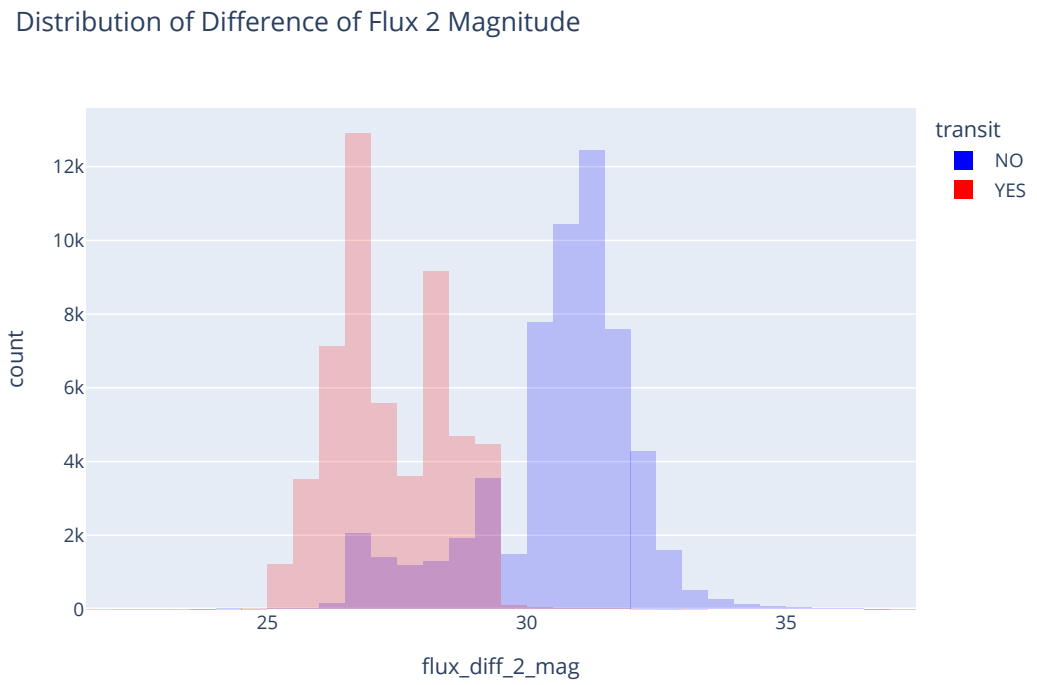


[44]:
```
fig = px.histogram(
        shuffled_tuc47_lc_df,
        x= 'flux_diff_2_err',
        color='transit',
        barmode='overlay',
        nbins=50,
        title=f"Distribution of Difference of Flux 2 Error",
        color_discrete_map=custom_colors,
        opacity=0.2
    )
fig.show()
```

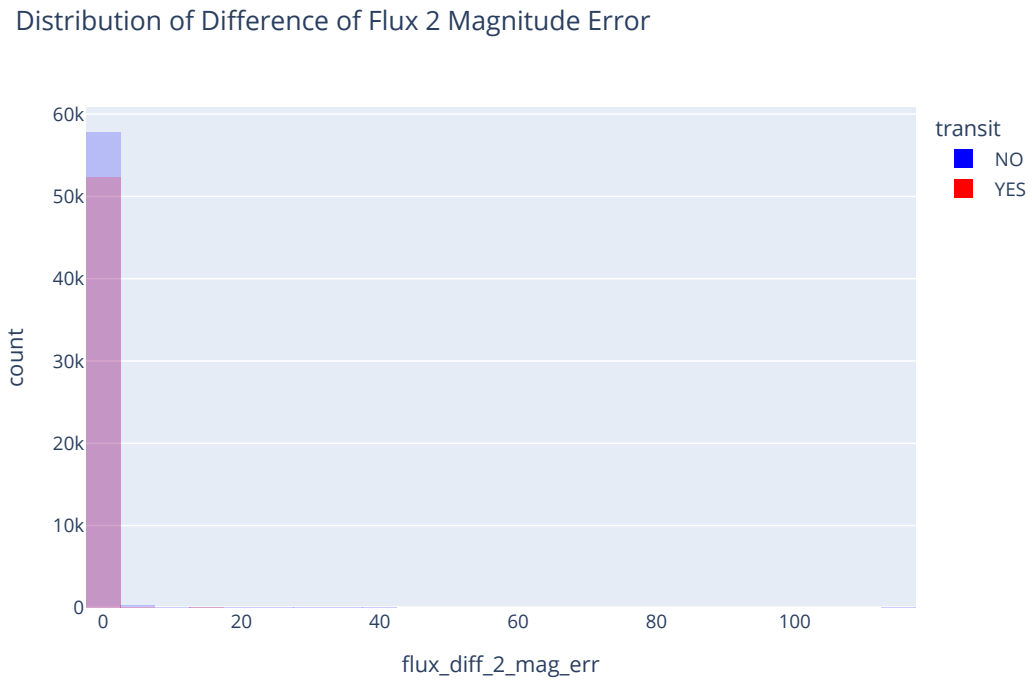


[45]:
```
fig = px.histogram(
        shuffled_tuc47_lc_df,
        x= 'flux_diff_2_mag',
        color='transit',
        barmode='overlay',
        nbins=50,
        title=f"Distribution of Difference of Flux 2 Magnitude",
        color_discrete_map=custom_colors,
        opacity=0.2
```

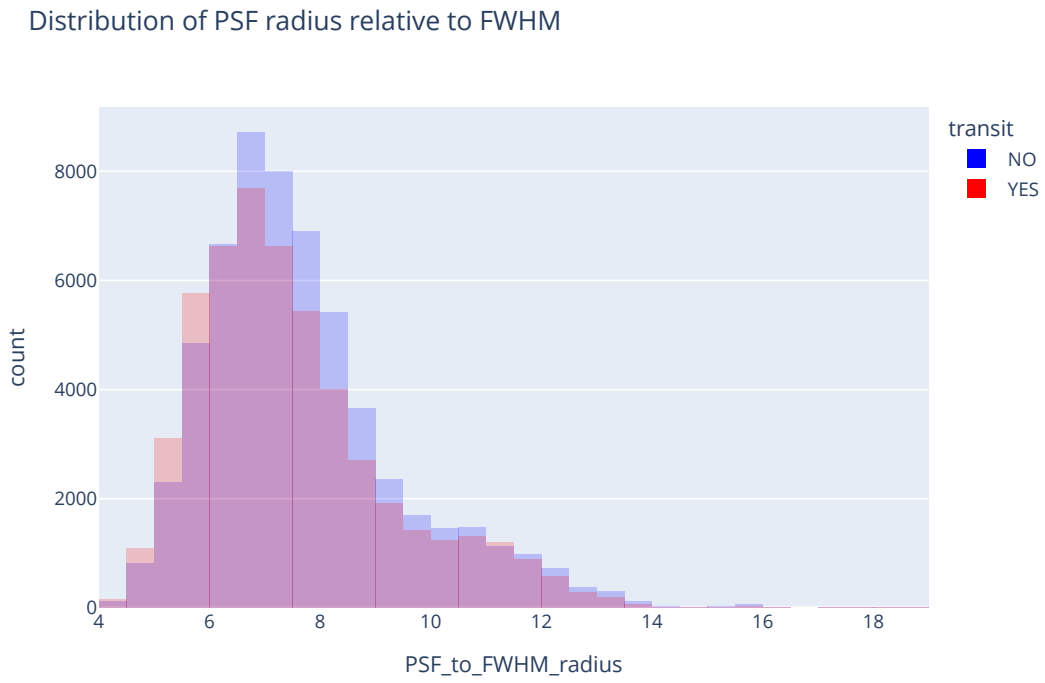
```
    )
fig.show()
```



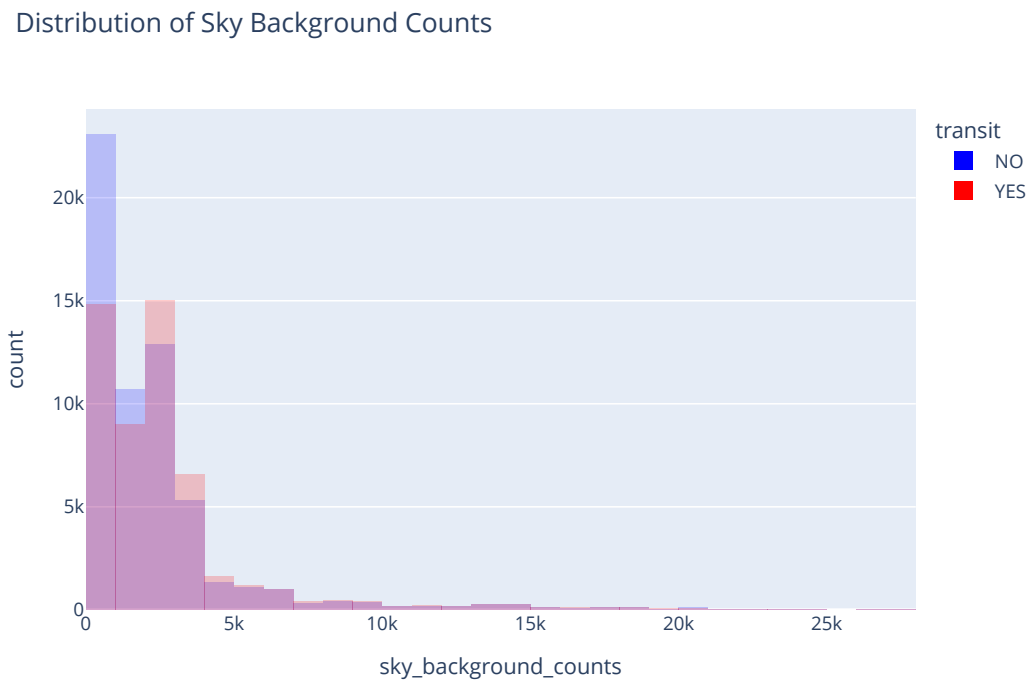
[46]:
```
fig = px.histogram(
        shuffled_tuc47_lc_df,
        x= 'flux_diff_2_mag_err',
        color='transit',
        barmode='overlay',
        nbins=50,
        title=f"Distribution of Difference of Flux 2 Magnitude Error",
        color_discrete_map=custom_colors,
        opacity=0.2
    )
fig.show()
```



[47]:
```
fig = px.histogram(
        shuffled_tuc47_lc_df,
        x= 'PSF_to_FWHM_radius',
        color='transit',
        barmode='overlay',
        nbins=50,
        title=f"Distribution of PSF radius relative to FWHM",
        color_discrete_map=custom_colors,
        opacity=0.2
    )
fig.show()
```



[48]:
```
fig = px.histogram(
        shuffled_tuc47_lc_df,
        x= 'sky_background_counts',
        color='transit',
        barmode='overlay',
        nbins=50,
        title=f"Distribution of Sky Background Counts",
        color_discrete_map=custom_colors,
        opacity=0.2
    )
fig.show()
```



Show the light curve of a specfifc star Let's see what a light curve actually looks like with our data. Since our data isn't phase-folded, there isn't an obvious dip like with other light curves. The first one is a real light curve with no transit in blue and the second in red is a light curve with an injected transit. I also plotted the airmass vs. Julian date as that's a common practice for this type of analysis.

[49]:
```
# 1. Get a list of all unique file names in the dataframe
unique_files = r_filt_df['source_file'].unique()

# 2. Select which file to plot (e.g., the first one at index 0)
target_file = unique_files[0]
print(f"Plotting data for: {target_file}")

# 3. Filter the dataframe to only include rows for that specific file
single_lc = r_filt_df[r_filt_df['source_file'] == target_file]
single_lc['MJD']-= 2458664.0

# 4. Plot
plt.figure(figsize=(10, 6))
plt.scatter(single_lc['MJD'], single_lc['flux_diff_2_mag'], marker='o', s=10,␣
 ↪color='blue')
```

```
plt.gca().invert_yaxis()
plt.xlim(744.75,745)

plt.xlabel('Time (MJD)')
plt.ylabel('Flux Magnitude (flux_diff_2_mag)')
plt.title(f'Light Curve: {target_file}') # Adds filename to title

plt.grid(True, alpha=0.3)
plt.legend()
plt.tight_layout()
plt.show()

plt.figure(figsize=(10, 2))
plt.scatter(single_lc['MJD'], single_lc['airmass'], marker = 'o', s=10,␣
 ↪color='blue')

plt.gca().invert_yaxis()
plt.xlim(744.75,745)

plt.ylabel('Airmass')
plt.xlabel('Time (MJD)')
plt.title(f'Light Curve: {target_file}')

plt.grid(True, alpha=0.3)
plt.tight_layout()
plt.show()
```

```
Plotting data for:
MISHAPS_F47T_S11_r_2019to2022/71/MISHAPS_F47T_S11_01030171.r.lc
```

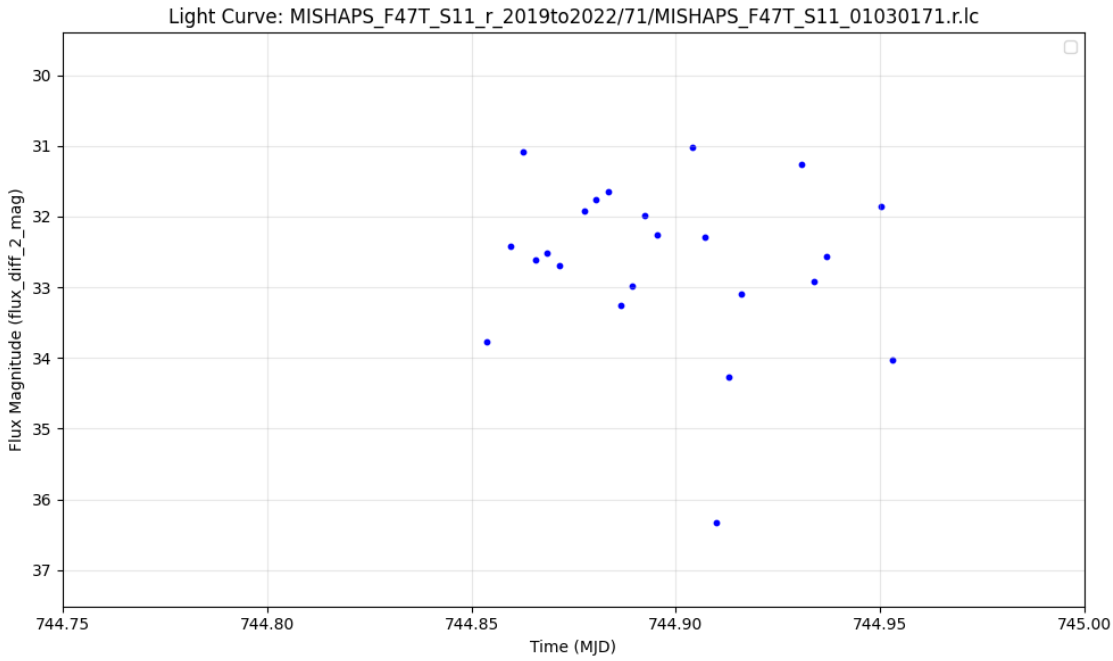
```
/tmp/slurmtmp.42555473/ipykernel_3595893/2972685161.py:10:
SettingWithCopyWarning:

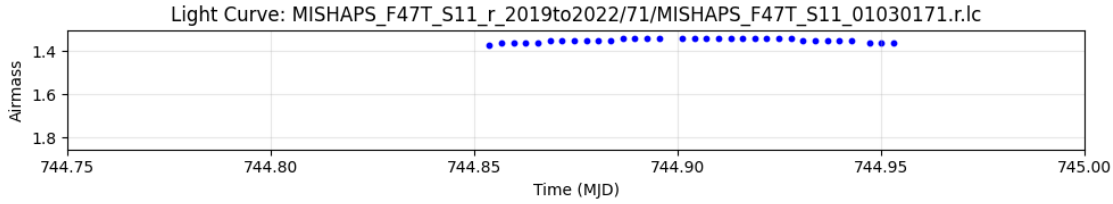

A value is trying to be set on a copy of a slice from a DataFrame.
Try using .loc[row_indexer,col_indexer] = value instead

See the caveats in the documentation: https://pandas.pydata.org/pandas-
docs/stable/user_guide/indexing.html#returning-a-view-versus-a-copy

/tmp/slurmtmp.42555473/ipykernel_3595893/2972685161.py:24: UserWarning:

No artists with labels found to put in legend.  Note that artists whose label
start with an underscore are ignored when legend() is called with no argument.
```





[50]:
```
# 1. Get a list of all unique file names in the dataframe
unique_files = injection_df['source_file'].unique()

# 2. Select which file to plot (e.g., the first one at index 0)
target_file = unique_files[0]
print(f"Plotting data for: {target_file}")

# 3. Filter the dataframe to only include rows for that specific file
single_lc = injection_df[injection_df['source_file'] == target_file]
single_lc['MJD']-= 2458664.0
# 4. Plot
plt.figure(figsize=(10, 6))
plt.scatter(single_lc['MJD'], single_lc['flux_diff_2_mag'],
            marker='o', s=10, color='red', label='Observed Flux')

plt.gca().invert_yaxis()
```

```
plt.xlim(744.75,745)

plt.xlabel('MJD (Time)')
plt.ylabel('Flux (flux_diff_2)')
plt.title(f'Light Curve: {target_file}') # Adds filename to title

plt.grid(True, alpha=0.3)
plt.legend()
plt.tight_layout()
plt.show()

plt.figure(figsize=(10, 2))
plt.scatter(single_lc['MJD'], single_lc['airmass'], marker = 'o', s=10,␣
 ↪color='red')

plt.gca().invert_yaxis()
plt.xlim(744.75,745)

plt.ylabel('Airmass')
plt.xlabel('Time (MJD)')
plt.title(f'Light Curve: {target_file}')

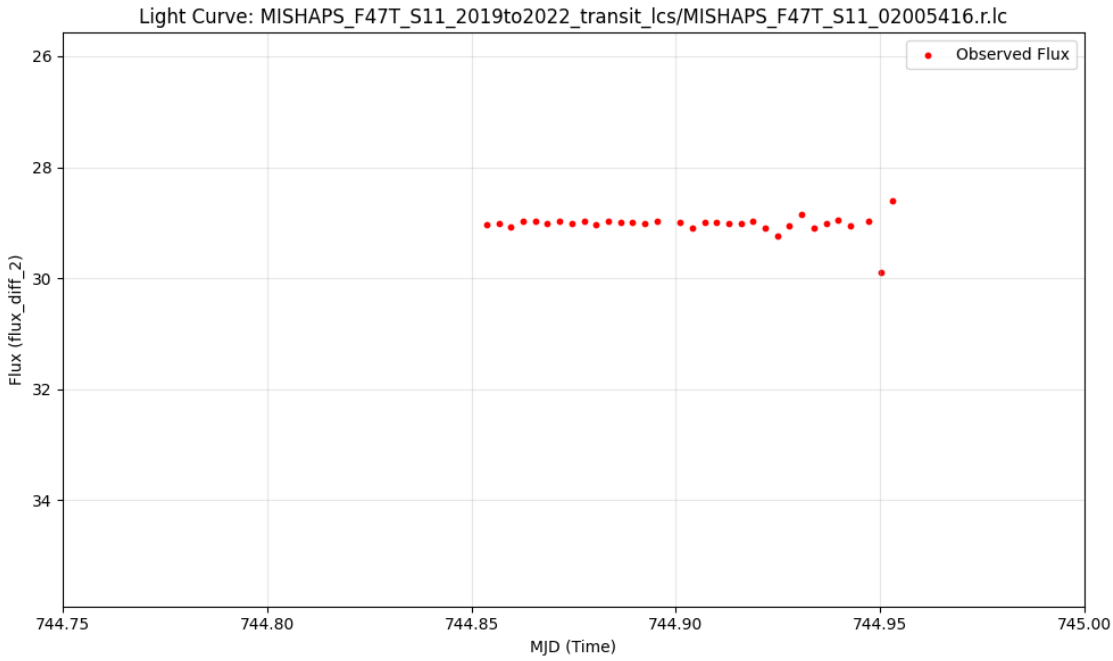
plt.grid(True, alpha=0.3)
plt.tight_layout()
plt.show()
```

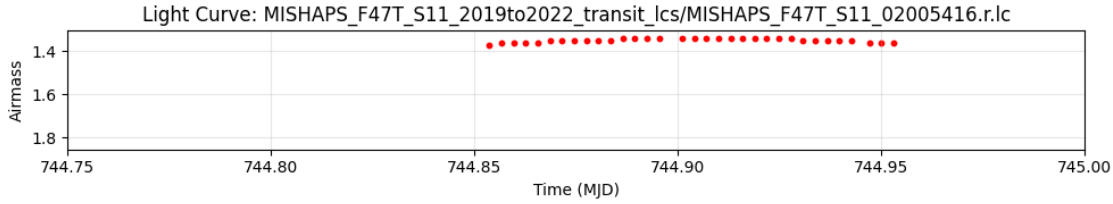
```
Plotting data for:
MISHAPS_F47T_S11_2019to2022_transit_lcs/MISHAPS_F47T_S11_02005416.r.lc
```

```
/tmp/slurmtmp.42555473/ipykernel_3595893/1591212913.py:10:
SettingWithCopyWarning:


A value is trying to be set on a copy of a slice from a DataFrame.
Try using .loc[row_indexer,col_indexer] = value instead

See the caveats in the documentation: https://pandas.pydata.org/pandas-
docs/stable/user_guide/indexing.html#returning-a-view-versus-a-copy
```





1.8 6. Methods

1.8.1 6.1. Method 1: Random Forest

The method that I will use is a random forest. A random forest is made up of multiple decision trees, which is a type of supervised learning algorithm typically used for classification in binary situations, in our case, or in multi-class situations. Using a random forest instead of a single decision tree allows us to overcome the two main drawbacks of decision trees, including its inclination to overfit data and its sensitivity to the actual distribution of the training data. Additionally, by using a random forest, we can learn exactly what physical characteristic that our model is using to determine transits due to the forest providing a clear feature importance ranking. This is extremely necessary for scientific validation. Since light curves tend to contain instrumental noise and stellar variability, using a random forest will reduce the chances of transits being detected due to random noise spikes because the forest can be made up of hundred, even thousands of trees. Some more advantages of using a random forest include its efficiency with smaller data sets and its ability to handle non-linear relationships in the data.

There are a couple of important drawbacks to consider as well. One of the biggest is the loss of temporal context. To use a random forest, the light curve must be summarized into statistics, including the mean, standard deviation, etc., which causes the model to lose the understanding of time. It will not be able to tell the difference between a dip that happens all at once, a transit, versus random low points scattered throughout the curve unless it is created with features that capture that consecutive behavior. Another drawbacks include feature dependency and sensitivity to imbalanced data.

Here are the steps I will take to train my random forest: 1) **Prepare features and labels.** The features that I will be looking at specifically are flux_diff_2, flux_diff_2_err, aobc, sky_flux_2, flux_diff_2_mag_err, PSF_to_FWHM_radius, PSF_ellipticity, airmass. The target labels are 1 for 'YES' and 0 for 'NO'. 2) **Split data for training/test.** Shuffle the dataset (done in the previous section), divide the data using `train_test_split` or `StratifiedKFold` while preseving the proportion of real vs. synthetic light curves, and create feature matrices and label vectors. 3) **Define the `RandomForestClassifier`.** Use parameters such as `n_estimators`, `max_depth`, `min_samples_leaf`, `class_weight='balanced'`, and `max_features='sqrt'`. 4) **Train/evaluate it with k-fold cross validation.** Use `k=5` to ensure balanced class representation. 5) **Explore hyperparamters.** Loop over a range of `max_depth` values (1–19) to see how tree depth affects performance. 6) **Analyze performance.** Plot the error measurements to check for overfitting.

The code that I am using is based off of module_4_2_random_forest from class. Below is the code that I will be directly pulling from and altering. **Do not run this code as it will not work.**

[ ]:
```
# This is the feature and label data for the full combined sample
dfCombined_features = dfCombined.iloc[:,:8]

# Get the labels
dfCombined_labels = dfCombined['class']
#
# Get our test and train data
df_train_digits,df_test_digits = train_test_split(dfCombined, test_size=0.2,␣
 ↪random_state=42)

# Get the features
df_train_features = df_train_digits.iloc[:,:8]

# Get the labels
df_train_labels = df_train_digits['class']

#
# Get the features of the test set
df_test_features = df_test_digits.iloc[:,:8]

#
# Get the labels of the test set
df_test_labels = df_test_digits['class']

kfolds = 5
```

```
skf = StratifiedKFold(n_splits=kfolds)

estimator = RandomForestClassifier(n_estimators=100,␣
 ↪max_depth=6,random_state=42)

avg_precision_train = 0.0
avg_recall_train = 0.0
avg_auc_train = 0.0
avg_precision_test = 0.0
avg_recall_test = 0.0
avg_auc_test = 0.0
numSplits = 0.0
#
# Now loop
for train_index, test_index in skf.split(dfCombined_features,dfCombined_labels):
    print("Training; loop counter",numSplits)
    numSplits += 1
    df_train_features = dfCombined_features.iloc[train_index]
    df_train_labels = dfCombined_labels.iloc[train_index]
    df_test_features = dfCombined_features.iloc[test_index]
    df_test_labels = dfCombined_labels.iloc[test_index]
#
# Now fit to our training set
    labels =[1,0]
    results_train,results_test =␣
 ↪run_fitter(estimator,df_train_features,df_train_labels,df_test_features,df_test_labels,labe

    avg_precision_train += results_train['precision']
    avg_recall_train += results_train['recall']
    avg_auc_train += results_train['auc']
#
    avg_precision_test += results_test['precision']
    avg_recall_test += results_test['recall']
    avg_auc_test += results_test['auc']
#
avg_precision_train /= numSplits
avg_recall_train /= numSplits
avg_auc_train /= numSplits
avg_precision_test /= numSplits
avg_recall_test /= numSplits
avg_auc_test /= numSplits
#
# Now print
print("Precision train/test␣
 ↪",round(avg_precision_train,3),round(avg_precision_test,3))
print("Recall train/test   ␣
 ↪",round(avg_recall_train,3),round(avg_recall_test,3))
```

```
print("AUC train/test       ",round(avg_auc_train,3),round(avg_auc_test,3))

# Create a list to store our results
rows = []

for max_depth in range(1,20):
    print("training with max depth =",max_depth)
    max_features = 8-2
    estimator =␣
 ↪RandomForestClassifier(n_estimators=100,random_state=42,max_depth=max_depth,max_features=ma
    avg_precision_train = 0.0
    avg_recall_train = 0.0
    avg_f1_score_train = 0.0
    avg_auc_train = 0.0
    avg_precision_test = 0.0
    avg_recall_test = 0.0
    avg_auc_test = 0.0
    avg_f1_score_test = 0.0
    numSplits = 0.0
#
# Now loop
    numSplits = 0
    for train_index, test_index in skf.
 ↪split(dfCombined_features,dfCombined_labels):
        print("Training; loop counter",numSplits)
        numSplits += 1
        df_train_features = dfCombined_features.iloc[train_index]
        df_train_labels = dfCombined_labels.iloc[train_index]
        df_test_features = dfCombined_features.iloc[test_index]
        df_test_labels = dfCombined_labels.iloc[test_index]
#
# Now fit to our training set
        labels =[1,0]
        results_train,results_test =␣
 ↪run_fitter(estimator,df_train_features,df_train_labels,df_test_features,df_test_labels,labe

        avg_precision_train += results_train['precision']
        avg_recall_train += results_train['recall']
        avg_auc_train += results_train['auc']
        avg_f1_score_train += results_train['f1_score']
#
        avg_precision_test += results_test['precision']
        avg_recall_test += results_test['recall']
        avg_auc_test += results_test['auc']
        avg_f1_score_test += results_test['f1_score']
#
    avg_precision_train /= numSplits
```

```
    avg_recall_train /= numSplits
    avg_auc_train /= numSplits
    avg_f1_score_train /= numSplits

    avg_precision_test /= numSplits
    avg_recall_test /= numSplits
    avg_auc_test /= numSplits
    avg_f1_score_test /= numSplits
#
# Fill dataframe
    rows.append({
        'max_depth':max_depth,
        'trainError_pre':1.0-avg_precision_train,'testError_pre':1.
 ↪0-avg_precision_test,
        'trainError_rec':1.0-avg_recall_train,'testError_rec':1.
 ↪0-avg_recall_test,
        'trainError_f1':1.0-avg_f1_score_train,'testError_f1':1.
 ↪0-avg_f1_score_test,
        'trainError_auc':1.0-avg_auc_train,'testError_auc':1.0-avg_auc_test
        })
#
#
# Convert our list of dictionaries to a dataframe
dfError = pd.DataFrame(rows)

# Now print
print(dfError.head)
```

1.9 7. Results

Experimental Setup: Using a random forest classifier, I determined whether the light curves of 47 Tucanae contained a transit event or not. The dataset consisted of both real and synthetic light curves, with features derived from flux differences, image similarity, sky background, PSF properties, and observational parameters. Each row corresponded to a single star, with the target label indicating the presence or absence of a transit.

Hyperparamters: The random forest classifier was configured with the following final hyperparameters,

`n_estimators` = 500 (number of trees)

`max_depth` = 7 (maximum depth of each tree) * This value was determined in the final evaluation

`min_samples_leaf` = 10 (minimum number of samples required to create a leaf node)

`class_weight` = 'balanced' (to compensate for the imbalance between transit and non-transit examples)

`max_features` = 'sqrt' (number of features considered at each split)

These hyperparameters were selected through a combination of manual tuning and k-fold cross-validation, exploring different maximum tree depths and evaluating performance metrics on the test folds to identify settings where the model's performance plateaued without overfitting.

Cross-Validation: I employed Stratified K-Fold cross-validation with 5 folds. Stratification ensured that each fold contained approximately the same proportion of transit and non-transit samples as the overall dataset. For each fold, the classifier was trained on 4/5 of the data and tested on the remaining 1/5. Metrics were calculated for both the training and test sets, and averaged across folds to provide strong estimates of model performance.

Evaluation Metrics: To evaluate the model, I used the following primary metrics, 1) Precision - the fraction of identified positives which are actually positive. 2) Recall or sensitivty - the fraction of all actual positives in our data that are identified as positive. 3) F-1 Score - the harmonic mean of precision and recall. 4) Area under the ROC curve (AUC) - AUC evaluates the model's ability to discriminate between classes across all classification thresholds. It is computed from the true positive rate (TPR) and false positive rate (FPR).

These metrics were computed for each fold during cross-validation, and their average values were reported for both training and test sets to assess model performance and generalization.

1.9.1 7.1. Train and Evaluate Models

This function will help with calculating and visualizing the results of the random forest. This was taken from the module_4_2_random_forest inclass module.

[51]:
```
def calc_performance(y_vals_true, y_vals_pred, labels,score):
#
# Get the numbers for the confusion matrix
# To get output: cf_matrix[true_label,pred_label]
    cf_matrix = confusion_matrix(y_vals_true, y_vals_pred, labels=labels)
    TP = cf_matrix[0,0]
    FN = cf_matrix[0,1]
    FP = cf_matrix[1,0]
    TN = cf_matrix[1,1]

#
# This is a graphic
    cf_disp =␣
 ↪ConfusionMatrixDisplay(confusion_matrix=cf_matrix,display_labels=labels)
#
    table = [
        [ "", "Predicted Class 1", "Precicted Class 0"],
        [ "True Class 1", TP, FN ],
        [ "True Class 0", FP, TN ]
    ]
    print_table_type='fancy_grid'
    print_table = tabulate(table, headers='firstrow', tablefmt=print_table_type)
#
    precision = TP / (TP + FP)
```

```
    recall = TP / (TP + FN)
    f1_score = 2.0 / ( (1.0/precision) + (1.0/recall) )
#
# Now get the ROC curve data
    fpr, tpr, threshold = metrics.roc_curve(y_vals_true, score, pos_label=1)
#
# Now get the AUC
    auc_val = auc(fpr,tpr)
#
# Return the matrix as part of a dictionary.   We could have just returned the
# dictionary itself, but we will be modifying this method and a dictionary makes
# it easy to return mtuliple objects in a organized way
    results = {'confusion_matrix_data':cf_matrix,
                  'confusion_matrix_display':cf_disp,
                  'confusion_matrix_print_table':print_table,
               'precision': precision,
               'recall': recall,
               'f1_score': f1_score,
               'fpr':fpr,
               'tpr':tpr,
               'threshold': threshold,
               'auc': auc_val}
    return results
```

This function helps to train a classifier, make predictions, and evaluate performance for both training and test sets. This was also taken from the module_4_2_random_forest inclass module.

[52]:
```
def␣
↪run_fitter(estimator,df_train_features,df_train_labels,df_test_features,df_test_labels,labe
↪


    # Actually do the training
    estimator.fit(df_train_features,df_train_labels)
    #
    # Now predict the classes for our train set
    y_train_pred = estimator.predict(df_train_features)
    #
    # Now predict the classes for our test set
    y_test_pred = estimator.predict(df_test_features)
    #
    # Get the results of the decision function for each digit
    y_train_score = estimator.predict_proba(df_train_features)[:,1]
    y_test_score = estimator.predict_proba(df_test_features)[:,1]
    #
    # Get resulst for train
    results_train =␣
↪calc_performance(df_train_labels,y_train_pred,labels,y_train_score)
```

```
    #
    # Get resulst for test
    results_test =␣
 ↪calc_performance(df_test_labels,y_test_pred,labels,y_test_score)
    #
    return results_train,results_test
```

This is the amount of data that we are working with. Since it looks like there is more background than signal, our sample will be somewhat biased. This will be accounted for during the training of the random forest, but it shouldn't cause too much of an issue since the discrepancy isn't that big.

[55]:
```
print("Length of signal sample:     ",len(injection_df))
print("Length of background sample: ",len(real_df))
print("Size of combined sample ",len(shuffled_tuc47_lc_df))
```

```
Length of signal sample:      53355
Length of background sample:  61248
Size of combined sample  110523
```

Get the features and label data for our sample

[66]:
```
# This is the feature and label data for the full combined sample
shuffled_tuc47_lc_df_features = shuffled_tuc47_lc_df[['flux_diff_2',␣
 ↪'flux_diff_2_err', 'aobc', 'sky_flux_2',
                                                      'flux_diff_2_mag_err',␣
 ↪'PSF_to_FWHM_radius', 'PSF_ellipticity','airmass']]

# Get the labels
shuffled_tuc47_lc_df_labels = shuffled_tuc47_lc_df['transit_value']
#
# Get our test and train data
df_train_digits, df_test_digits = train_test_split(shuffled_tuc47_lc_df,␣
 ↪test_size=0.2, random_state=42)

# Get the features
df_train_features = df_train_digits[['flux_diff_2', 'flux_diff_2_err', 'aobc',␣
 ↪'sky_flux_2',
                                     'flux_diff_2_mag_err',␣
 ↪'PSF_to_FWHM_radius', 'PSF_ellipticity','airmass']]

# Get the labels
df_train_labels = df_train_digits['transit_value']

#
# Get the features of the test set
df_test_features = df_test_digits[['flux_diff_2', 'flux_diff_2_err', 'aobc',␣
 ↪'sky_flux_2',
```

```
                                    'flux_diff_2_mag_err', 'PSF_to_FWHM_radius',␣
 ↪'PSF_ellipticity','airmass']]

#
# Get the labels of the test set
df_test_labels = df_test_digits['transit_value']
```

Train a single random forest using k-fold validation

[67]:
```
kfolds = 5
skf = StratifiedKFold(n_splits=kfolds)

estimator = RandomForestClassifier(n_estimators=500, max_depth=6,␣
 ↪random_state=42)

avg_precision_train = 0.0
avg_recall_train = 0.0
avg_auc_train = 0.0
avg_precision_test = 0.0
avg_recall_test = 0.0
avg_auc_test = 0.0
numSplits = 0.0

# Now loop
for train_index, test_index in skf.split(shuffled_tuc47_lc_df_features,␣
 ↪shuffled_tuc47_lc_df_labels):
    print("Training; loop counter",numSplits)
    numSplits += 1
    df_train_features = shuffled_tuc47_lc_df_features.iloc[train_index]
    df_train_labels = shuffled_tuc47_lc_df_labels.iloc[train_index]
    df_test_features = shuffled_tuc47_lc_df_features.iloc[test_index]
    df_test_labels = shuffled_tuc47_lc_df_labels.iloc[test_index]
#
# Now fit to our training set
    labels =[1,0]
    results_train, results_test = run_fitter(estimator, df_train_features,␣
 ↪df_train_labels, df_test_features, df_test_labels, labels)

    avg_precision_train += results_train['precision']
    avg_recall_train += results_train['recall']
    avg_auc_train += results_train['auc']
#
    avg_precision_test += results_test['precision']
    avg_recall_test += results_test['recall']
    avg_auc_test += results_test['auc']
#
avg_precision_train /= numSplits
```

```
avg_recall_train /= numSplits
avg_auc_train /= numSplits
avg_precision_test /= numSplits
avg_recall_test /= numSplits
avg_auc_test /= numSplits
#
# Now print
print("Precision train/test␣
  ↪",round(avg_precision_train,3),round(avg_precision_test,3))
print("Recall train/test   ␣
  ↪",round(avg_recall_train,3),round(avg_recall_test,3))
print("AUC train/test       ",round(avg_auc_train,3),round(avg_auc_test,3))
```

```
Training; loop counter 0.0
Training; loop counter 1.0
Training; loop counter 2.0
Training; loop counter 3.0
Training; loop counter 4.0
Precision train/test  0.849 0.848
Recall train/test     0.963 0.962
AUC train/test        0.956 0.955
```

Visualize our results using the calc_performance function defined above and determine which feature contributed the most

[68]:
```
print(results_test['confusion_matrix_print_table'])
```

```
                Predicted Class 1     Precicted Class 0

 True Class 1                10109                   363

 True Class 0                 1845                  9787
```

[69]:
```
importances = pd.Series(estimator.feature_importances_,␣
  ↪index=shuffled_tuc47_lc_df_features.columns).sort_values(ascending=False)
print(importances)
```

```
flux_diff_2_mag_err    0.798531
flux_diff_2_err        0.130324
flux_diff_2            0.023703
sky_flux_2             0.023345
PSF_to_FWHM_radius     0.009913
aobc                   0.006910
PSF_ellipticity        0.005147
airmass                0.002126
dtype: float64
```

Explore the hyperparameters by looping over a range of `max_depth` values

[72]:
```
# Create a list to store our results
rows = []

for max_depth in range(1,20):
    print("training with max depth =",max_depth)
    # Adjusted Estimator
    estimator = RandomForestClassifier(n_estimators=500,
                                       random_state=42,
                                       max_depth=max_depth,              #␣
 ↪Restrict depth to where test error plateaued
                                       min_samples_leaf=10,      # Requires 10␣
 ↪samples to create a leaf (reduces noise)
                                       class_weight='balanced',  # Crucial for␣
 ↪transit detection
                                       max_features='sqrt'       # Standard␣
 ↪default, usually better than hardcoding 8-2
)
    avg_precision_train = 0.0
    avg_recall_train = 0.0
    avg_f1_score_train = 0.0
    avg_auc_train = 0.0
    avg_precision_test = 0.0
    avg_recall_test = 0.0
    avg_auc_test = 0.0
    avg_f1_score_test = 0.0
    numSplits = 0.0
#
# Now loop
    numSplits = 0
    for train_index, test_index in skf.split(shuffled_tuc47_lc_df_features,␣
 ↪shuffled_tuc47_lc_df_labels):
        print("Training; loop counter",numSplits)
        numSplits += 1
        df_train_features = shuffled_tuc47_lc_df_features.iloc[train_index]
        df_train_labels = shuffled_tuc47_lc_df_labels.iloc[train_index]
        df_test_features = shuffled_tuc47_lc_df_features.iloc[test_index]
        df_test_labels = shuffled_tuc47_lc_df_labels.iloc[test_index]
# Now fit to our training set
        labels =[1,0]
        results_train, results_test = run_fitter(estimator, df_train_features,␣
 ↪df_train_labels, df_test_features, df_test_labels,labels)

        avg_precision_train += results_train['precision']
        avg_recall_train += results_train['recall']
        avg_auc_train += results_train['auc']
        avg_f1_score_train += results_train['f1_score']
```

```
#
        avg_precision_test += results_test['precision']
        avg_recall_test += results_test['recall']
        avg_auc_test += results_test['auc']
        avg_f1_score_test += results_test['f1_score']
#
    avg_precision_train /= numSplits
    avg_recall_train /= numSplits
    avg_auc_train /= numSplits
    avg_f1_score_train /= numSplits

    avg_precision_test /= numSplits
    avg_recall_test /= numSplits
    avg_auc_test /= numSplits
    avg_f1_score_test /= numSplits
#
# Fill dataframe
    rows.append({
        'max_depth':max_depth,
        'trainError_pre':1.0-avg_precision_train,'testError_pre':1.
 ↪0-avg_precision_test,
        'trainError_rec':1.0-avg_recall_train,'testError_rec':1.
 ↪0-avg_recall_test,
        'trainError_f1':1.0-avg_f1_score_train,'testError_f1':1.
 ↪0-avg_f1_score_test,
        'trainError_auc':1.0-avg_auc_train,'testError_auc':1.0-avg_auc_test
        })
#
#
# Convert our list of dictionaries to a dataframe
dfError = pd.DataFrame(rows)

# Now print
print(dfError.head)
```

```
training with max depth = 1
Training; loop counter 0
Training; loop counter 1
Training; loop counter 2
Training; loop counter 3
Training; loop counter 4
training with max depth = 2
Training; loop counter 0
Training; loop counter 1
Training; loop counter 2
Training; loop counter 3
Training; loop counter 4
```

```
training with max depth = 3
Training; loop counter 0
Training; loop counter 1
Training; loop counter 2
Training; loop counter 3
Training; loop counter 4
training with max depth = 4
Training; loop counter 0
Training; loop counter 1
Training; loop counter 2
Training; loop counter 3
Training; loop counter 4
training with max depth = 5
Training; loop counter 0
Training; loop counter 1
Training; loop counter 2
Training; loop counter 3
Training; loop counter 4
training with max depth = 6
Training; loop counter 0
Training; loop counter 1
Training; loop counter 2
Training; loop counter 3
Training; loop counter 4
training with max depth = 7
Training; loop counter 0
Training; loop counter 1
Training; loop counter 2
Training; loop counter 3
Training; loop counter 4
training with max depth = 8
Training; loop counter 0
Training; loop counter 1
Training; loop counter 2
Training; loop counter 3
Training; loop counter 4
training with max depth = 9
Training; loop counter 0
Training; loop counter 1
Training; loop counter 2
Training; loop counter 3
Training; loop counter 4
training with max depth = 10
Training; loop counter 0
Training; loop counter 1
Training; loop counter 2
Training; loop counter 3
Training; loop counter 4
```

```
training with max depth = 11
Training; loop counter 0
Training; loop counter 1
Training; loop counter 2
Training; loop counter 3
Training; loop counter 4
training with max depth = 12
Training; loop counter 0
Training; loop counter 1
Training; loop counter 2
Training; loop counter 3
Training; loop counter 4
training with max depth = 13
Training; loop counter 0
Training; loop counter 1
Training; loop counter 2
Training; loop counter 3
Training; loop counter 4
training with max depth = 14
Training; loop counter 0
Training; loop counter 1
Training; loop counter 2
Training; loop counter 3
Training; loop counter 4
training with max depth = 15
Training; loop counter 0
Training; loop counter 1
Training; loop counter 2
Training; loop counter 3
Training; loop counter 4
training with max depth = 16
Training; loop counter 0
Training; loop counter 1
Training; loop counter 2
Training; loop counter 3
Training; loop counter 4
training with max depth = 17
Training; loop counter 0
Training; loop counter 1
Training; loop counter 2
Training; loop counter 3
Training; loop counter 4
training with max depth = 18
Training; loop counter 0
Training; loop counter 1
Training; loop counter 2
Training; loop counter 3
Training; loop counter 4
```

```
training with max depth = 19
Training; loop counter 0
Training; loop counter 1
Training; loop counter 2
Training; loop counter 3
Training; loop counter 4
<bound method NDFrame.head of     max_depth  trainError_pre  testError_pre
trainError_rec  testError_rec  \
0           1        0.164956       0.165041        0.054403       0.054871
1           2        0.164523       0.164732        0.050636       0.051185
2           3        0.163851       0.164145        0.045613       0.045857
3           4        0.160560       0.161077        0.040642       0.041349
4           5        0.158637       0.159387        0.036360       0.037377
5           6        0.155050       0.156359        0.033280       0.034607
6           7        0.148008       0.150669        0.030544       0.032907
7           8        0.142743       0.146446        0.027226       0.030673
8           9        0.138543       0.143666        0.023792       0.028515
9          10        0.134687       0.141680        0.020741       0.027541
10         11        0.131176       0.139557        0.018626       0.026834
11         12        0.126884       0.136721        0.017213       0.026261
12         13        0.122179       0.133769        0.016272       0.026146
13         14        0.118731       0.131699        0.015265       0.025688
14         15        0.115260       0.129697        0.014864       0.025803
15         16        0.111783       0.128066        0.014310       0.025841
16         17        0.108697       0.126733        0.013928       0.026070
17         18        0.106052       0.125333        0.013708       0.026032
18         19        0.103491       0.123694        0.013484       0.026127

    trainError_f1  testError_f1  trainError_auc  testError_auc
0        0.113113      0.113370        0.076299       0.076641
1        0.111213      0.111575        0.068127       0.068478
2        0.108636      0.108912        0.060991       0.061562
3        0.104599      0.105205        0.053702       0.054544
4        0.101640      0.102513        0.048453       0.049625
5        0.098257      0.099585        0.043868       0.045593
6        0.093064      0.095609        0.039348       0.041856
7        0.088630      0.092238        0.034050       0.037686
8        0.084750      0.089720        0.028344       0.033303
9        0.081234      0.088171        0.022856       0.029388
10       0.078325      0.086665        0.018375       0.026136
11       0.075289      0.084815        0.014945       0.023752
12       0.072239      0.083110        0.012418       0.021940
13       0.069866      0.081746        0.010452       0.020662
14       0.067757      0.080680        0.008936       0.019672
15       0.065582      0.079787        0.007767       0.018704
16       0.063705      0.079148        0.006901       0.018152
17       0.062149      0.078354        0.006271       0.017647
18       0.060639      0.077486        0.005746       0.017194  >
```

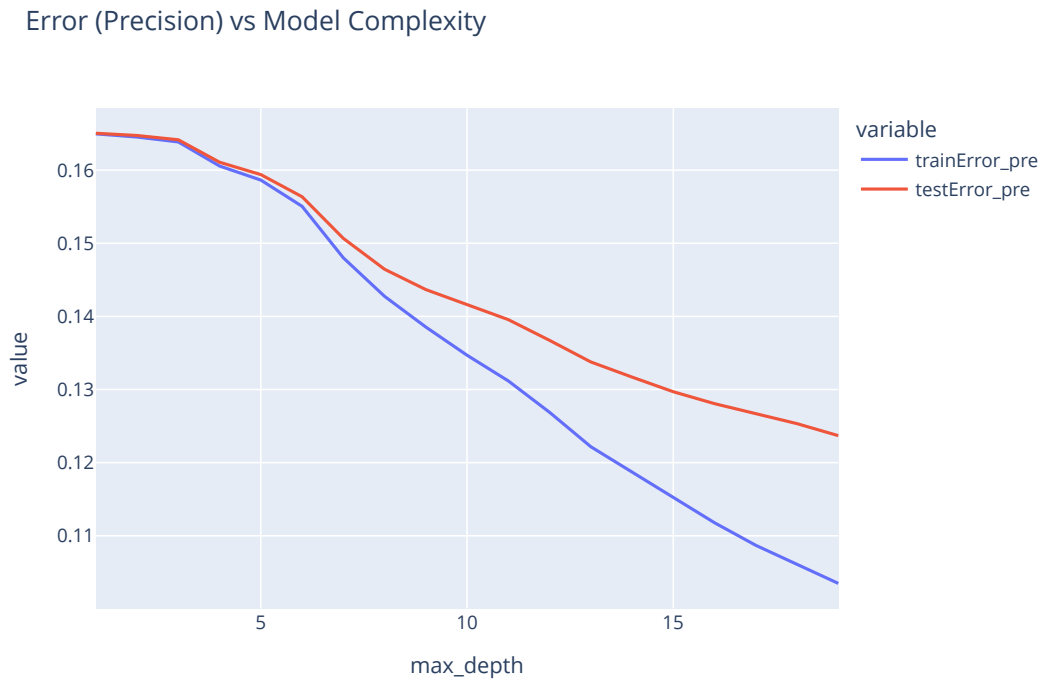
1.9.2 7.2. Visualize Results

[80]:
```
fig = px.line(dfError,x='max_depth', y=['trainError_pre','testError_pre'],
                      title='Error (Precision) vs Model Complexity')
fig.show()

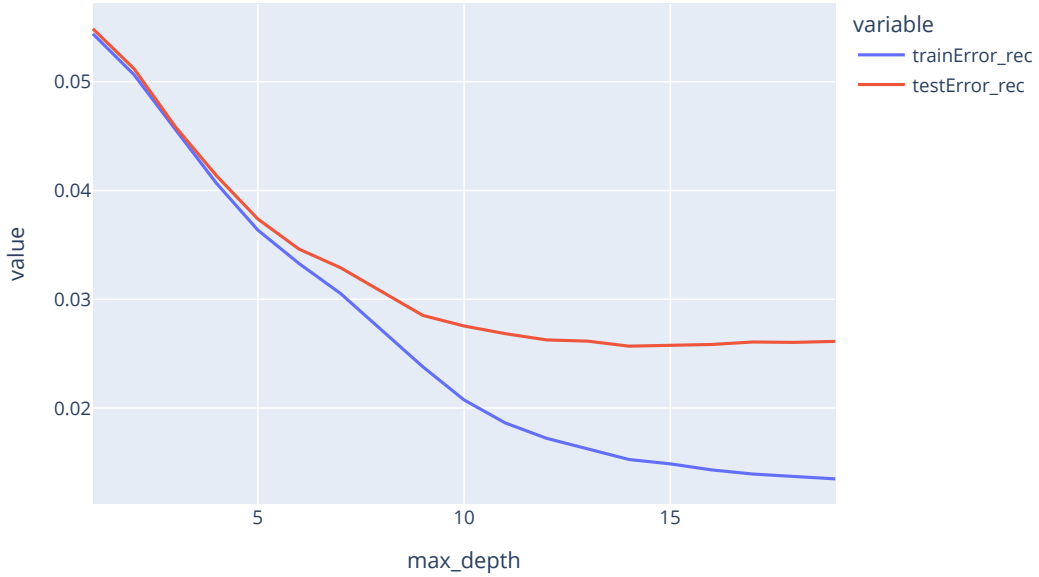
fig = px.line(dfError,x='max_depth', y=['trainError_rec','testError_rec'],
                      title='Error (Recall) vs Model Complexity')
fig.show()

fig = px.line(dfError,x='max_depth', y=['trainError_auc','testError_auc'],
                      title='Error (AUC) vs Model Complexity')
fig.show()

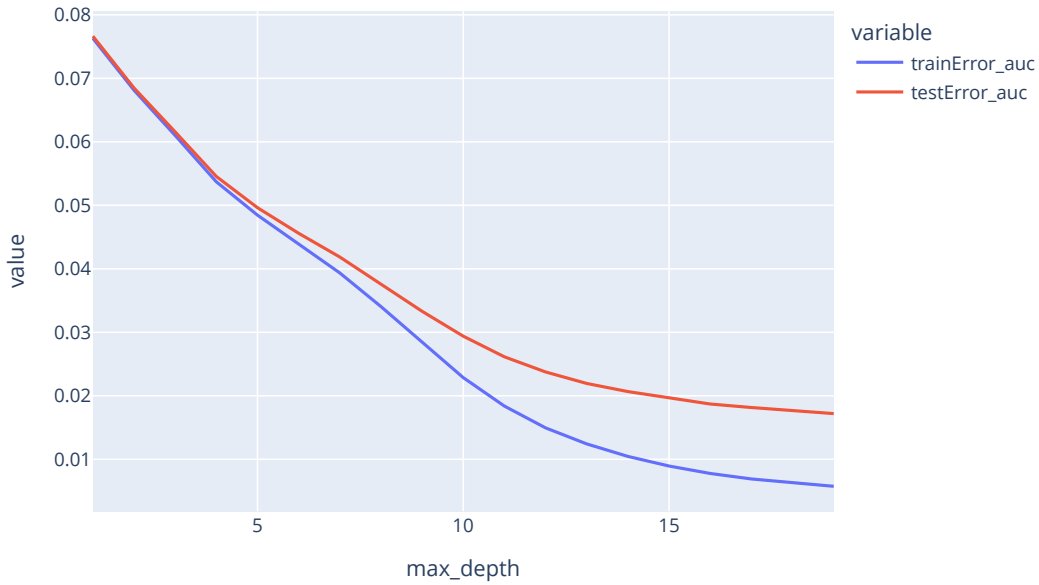
fig = px.line(dfError,x='max_depth', y=['trainError_f1','testError_f1'],
                      title='Error (F1-Score) vs Model Complexity')
fig.show()
```

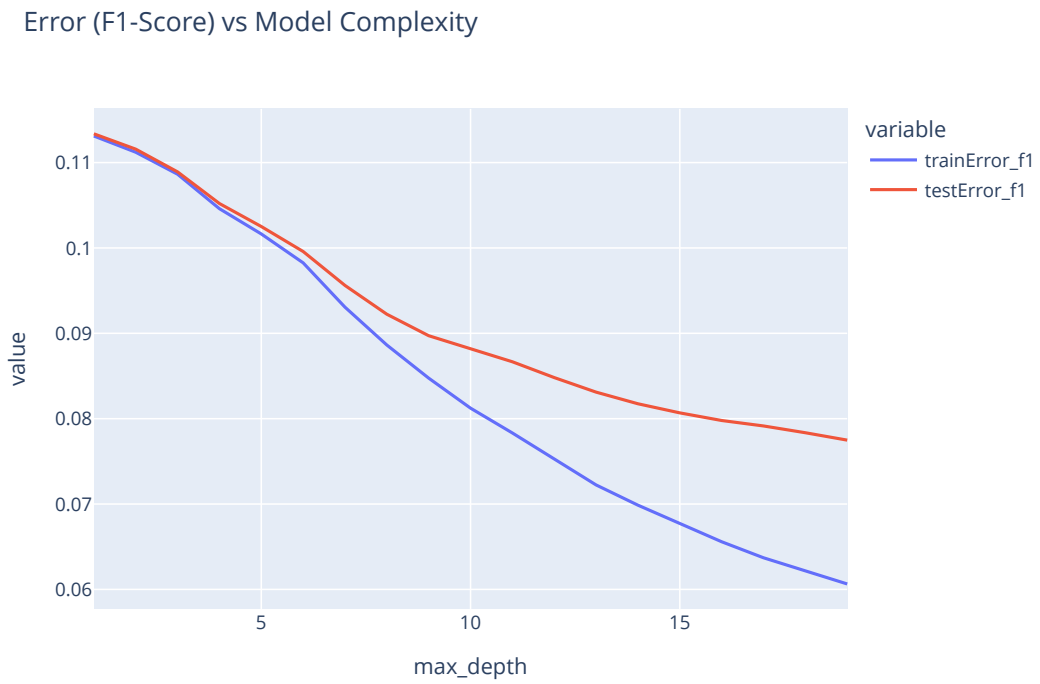


Error (Recall) vs Model Complexity



Error (AUC) vs Model Complexity





From this, we can say that the optimal `max_depth` is 7. So, we can retrain our random forest using this new parameter value and evalute it using the 5 fold StratifiedKFold that we did previously.

Final Evaluation

[77]:
```
kfolds = 5
skf = StratifiedKFold(n_splits=kfolds)

estimator = RandomForestClassifier(n_estimators=500, max_depth=7,␣
 ↪min_samples_leaf=10, class_weight='balanced', max_features='sqrt',
                                    random_state=42)

avg_precision_train = 0.0
avg_recall_train = 0.0
avg_auc_train = 0.0
avg_precision_test = 0.0
avg_recall_test = 0.0
avg_auc_test = 0.0
numSplits = 0.0
```

```
# Now loop
for train_index, test_index in skf.split(shuffled_tuc47_lc_df_features,␣
 ↪shuffled_tuc47_lc_df_labels):
    print("Training; loop counter",numSplits)
    numSplits += 1
    df_train_features = shuffled_tuc47_lc_df_features.iloc[train_index]
    df_train_labels = shuffled_tuc47_lc_df_labels.iloc[train_index]
    df_test_features = shuffled_tuc47_lc_df_features.iloc[test_index]
    df_test_labels = shuffled_tuc47_lc_df_labels.iloc[test_index]
#
# Now fit to our training set
    labels =[1,0]
    results_train, results_test = run_fitter(estimator, df_train_features,␣
 ↪df_train_labels, df_test_features, df_test_labels, labels)

    avg_precision_train += results_train['precision']
    avg_recall_train += results_train['recall']
    avg_auc_train += results_train['auc']
#
    avg_precision_test += results_test['precision']
    avg_recall_test += results_test['recall']
    avg_auc_test += results_test['auc']
#
avg_precision_train /= numSplits
avg_recall_train /= numSplits
avg_auc_train /= numSplits
avg_precision_test /= numSplits
avg_recall_test /= numSplits
avg_auc_test /= numSplits
#
# Now print
print("Precision train/test␣
 ↪",round(avg_precision_train,3),round(avg_precision_test,3))
print("Recall train/test   ␣
 ↪",round(avg_recall_train,3),round(avg_recall_test,3))
print("AUC train/test       ",round(avg_auc_train,3),round(avg_auc_test,3))
```

```
Training; loop counter 0.0
Training; loop counter 1.0
Training; loop counter 2.0
Training; loop counter 3.0
Training; loop counter 4.0
Precision train/test  0.852 0.849
Recall train/test     0.969 0.967
AUC train/test        0.961 0.958
```

Visualize results and determine feature importance

[79]:
```
print(results_test['confusion_matrix_print_table'])

importances_final = pd.Series(estimator.feature_importances_,␣
  ↪index=shuffled_tuc47_lc_df_features.columns).sort_values(ascending=False)
print(importances_final)
```

```
                Predicted Class 1    Precicted Class 0

 True Class 1               10166                  306

 True Class 0                1834                 9798

flux_diff_2_mag_err    0.799960
flux_diff_2_err        0.126621
flux_diff_2            0.024739
sky_flux_2             0.023155
PSF_to_FWHM_radius     0.010422
aobc                   0.007153
PSF_ellipticity        0.005885
airmass                0.002065
dtype: float64
```

1.10 8. Discussion

Interpretation of Results

The final Random Forest classifier achieved strong performance, with a test precision of 0.849, recall of 0.967, and an AUC of 0.958. These results indicate that the model is highly effective at identifying transit events while maintaining a relatively low false-positive rate. In particular, the high recall demonstrates that the majority of true transit signals are successfully recovered, which is critical in transit searches where missed detections are costly. However, high performance metrics do not imply that the problem is fully solved. Even with a high AUC and recall, false positives remain present, and model predictions must be interpreted probabilistically rather than as definitive classifications. Additionally, performance is measured on data drawn from the same distribution as the training set, meaning real-world generalization to unseen observing conditions or instrumental systematics is not guaranteed.

Error Analysis

Qualitative inspection of misclassified examples suggests that the model's failures are likely associated with light curves that exhibit noise patterns or systematics resembling transit-like features. False positives may arise from fluctuations in flux caused by variable sky background, PSF distortions, or atmospheric effects, which can mimic shallow transit signatures. On the other hand, false negatives may correspond to low signal-to-noise transits or events with atypical shapes that deviate from the injected transit models used during training. These patterns suggest that the model is sensitive to feature-level similarities between real noise and transit signals. A potential hypothesis is that additional time-domain or shape-based features, such as transit duration, asymmetry, or periodicity, could help reduce these uncertainties.

Comparison of Methods
This work focused on a random forest classifier due to its ability to handle non-linear feature interactions and smaller datasets without requiring feature scaling. Although alternative models were not extensively explored, random forests are particularly well suited for this problem because they can naturally handle diverse features derived from photometry, image statistics, and observing conditions. Future comparisons with models such as gradient-boosted trees, like in Gabruseva et al. (2019), or 1D convolutional neural networks may yield further performance gains, particularly if time-series–specific representations are incorporated. However, such models may require larger training sets and more careful regularization than what I currently have.

Overfitting
There was little evidence of overfitting observed in the final model evaluation. The training and test metrics closely matched across all evaluated measures, indicating good generalization. Overfitting was mitigated by restricting tree depth, enforcing a minimum number of samples per leaf, and using stratified k-fold cross-validation to ensure consistent class representation across folds. The hyperparameter exploration demonstrated that increasing tree depth beyond a certain point yielded diminishing returns, and selecting a maximum depth of 7 provided a favorable balance between bias and variance.

Limitations
There are several limitations should be noted for this work. First, the training dataset includes synthetic light curves with injected transits, which may not fully capture the complexity and diversity of real astrophysical transit signals. Second, the feature set is limited to summary statistics and image-derived quantities, rather than full time-series representations. This may restrict the model's ability to detect more subtle or irregular transit signatures. Additionally, the model's performance has not yet been validated on an entirely independent observational dataset, raising the possibility that the model could fail when applied to new data. Despite these limitations, the results demonstrate that random forest classifiers can effectively identify transit-like signals in light curve data and provide a strong foundation for future improvements.

1.11 9. Conclusions & Future Work

Summary
In this work, I developed and evaluated a machine learning approach for identifying transit events in light curves using a random forest classifier. The model was trained on a combination of real light curves and synthetic light curves with injected transit signals, using features derived from photometric measurements, sky background statistics, and PSF properties. Through systematic feature selection, hyperparameter tuning, and stratified k-fold cross-validation, the classifier demonstrated strong and stable performance across multiple evaluation metrics.

Future Work
There are several things that I would like to do to improve the transit classification performance of this random forest classifier and of other machine learning alternatives, like a 1D CNN. First, after processing the light curve data the same way I did with 47 Tucanae, I would run the Omega Centauri data through this classifier. This would be a true test to see how well this model performs on unseen data with no injected transits in the light curves. Second, finding ways to incorporate additional features, such as time-series shape descriptors, transit duration estimates, or periodicity metrics, may help reduce false positives and improve discrimination between genuine transits and

noise-induced fluctuations. With greater computational resources, more complex models such as gradient-boosted decision trees or 1D CNNs could be investigated to further speed up the process of classifying transit events using data obtained from multiple astronomical surveys.